



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**PROYECTO "APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR
LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS
CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE TACNA,
2026"**

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo

(2021070015)

Anampa Pancca David Jordan

(2022074268)

Tacna – Perú

2026



Índice

Contenido

Introducción	3
Guía de Laboratorio N° 01 – REVISION DE CODIGO	4
1. Información sobre el evento práctico	4
1.1. Objetivos.....	4
1.2. Equipos, materiales, programas y recursos	4
1.3. Seguridad.....	5
1. Desarrollo de la Práctica.....	5
2. Conclusiones.....	46

Introducción

La calidad del código constituye un aspecto fundamental en el desarrollo de software, debido a que influye directamente en la mantenibilidad, legibilidad, escalabilidad y confiabilidad de una aplicación. A medida que un proyecto crece, la ausencia de estándares y buenas prácticas puede generar problemas como duplicación de código, dificultad para realizar modificaciones, errores recurrentes y una mayor complejidad en el mantenimiento del sistema.

En el presente laboratorio se realiza una revisión y refactorización del código desarrollado para el proyecto “Aplicación móvil colaborativa para mejorar la efectividad en la asistencia de mascotas perdidas con participación de voluntarios en la ciudad de Tacna, 2026”, desarrollado en el curso de Construcción de Software II. La revisión se enfoca principalmente en los módulos relacionados con la autenticación de usuarios, el registro y visualización de avistamientos, y la gestión de comentarios en las publicaciones.

Para esta evaluación se consideran principios y buenas prácticas de construcción de software, tales como evitar la duplicación de código (DRY), utilizar comentarios únicamente cuando aporten información relevante, aplicar el principio Fail Fast, evitar números mágicos, emplear nombres descriptivos, reducir el uso de variables globales, retornar resultados en lugar de utilizar impresiones por consola y organizar adecuadamente los espacios en blanco. Asimismo, se revisan aspectos relacionados con las convenciones de nomenclatura y la responsabilidad de las variables y métodos.

A partir de los problemas identificados en el código original, se realizaron diferentes acciones de refactorización, entre ellas la creación de utilidades reutilizables, la centralización de procesos repetitivos, la incorporación de constantes descriptivas, la validación temprana de datos y archivos, la mejora de los nombres de variables y la reorganización del acceso a los datos almacenados en Firebase. Estas modificaciones buscan obtener un código más limpio, comprensible y fácil de mantener.

Finalmente, la práctica permite comprobar la importancia de realizar revisiones periódicas del código durante el desarrollo de un proyecto, ya que estas permiten detectar deficiencias antes de que se conviertan en problemas mayores y contribuyen a construir una aplicación más ordenada, estable y preparada para futuras mejoras.

Guía de Laboratorio N° 01 – REVISION DE CODIGO

1. Información sobre el evento práctico

1.1. Objetivos:

Objetivo general

- Realizar la revisión del código fuente de una aplicación de software, considerando buenas prácticas y estándares de construcción de software, con la finalidad de identificar errores, mejorar la calidad del código y facilitar su mantenimiento.

Objetivos específicos

- Identificar errores y posibles problemas en el código fuente.
- Verificar el cumplimiento de estándares y convenciones de programación.
- Evaluar la estructura, organización y legibilidad del código.
- Detectar código duplicado, innecesario o difícil de mantener.
- Comprobar el correcto uso de nombres para variables, métodos, clases y componentes.
- Proponer mejoras que permitan obtener un código más limpio, seguro y mantenible.
- Documentar las observaciones y recomendaciones encontradas durante la revisión.

1.2. Equipos, materiales, programas y recursos

- Computadora o laptop.
- Conexión a Internet, de ser necesaria.
- Software ofimático para la elaboración de la documentación.
- Software requerido para su aplicación
- Aplicación diseñada e implementada en el Curso de Construcción de Software II
- Formatos de despliegue para la documentación del Proyecto de

Software en proceso de implementación**1.3. Seguridad**

- Protocolo de Seguridad para laboratorios y talleres
- Protocolo ante la presencialidad en Laboratorios

1. Desarrollo de la Práctica

● Autenticar Usuario

- No te repitas.

En el código de autenticación, esta parte se ve claramente en la ruta principal de app.dart, donde hay muchas entradas repetidas con la misma estructura:

- cada ruta crea un provider
- cada ruta asigna una pantalla
- se repite el patrón de:
 - ChangeNotifierProvider(
 - create: (_) => XVM(),
 - child: const PantallaX(),
 -)

Eso es exactamente duplicación de código.

Qué se observa en tu código

1) Repetición de providers

En varias rutas haces algo similar:

- /login
- /registro
- /recuperar
- /perfil
- /inicio

Todos siguen la misma forma:

- crear un ViewModel
- enlazarlo con la pantalla
- devolver la pantalla

Eso se puede centralizar.

2) Repetición de pantalla de login

También aparece aquí:

- `"/login"` crea `LoginVM`
- `"/inicio"` también crea `LoginVM` dentro de `loginScreen`

Esto significa que la misma lógica se está creando más de una vez.

3) Repetición de notificaciones

También se repite esta parte:

- `"/inicio"` usa `NotificacionVM()..escucharNotificaciones()`
- `"/notificaciones"` usa `NotificacionVM()..escucharNotificaciones()`

Entonces el mismo patrón se construye dos veces.

```
71      }, // ThemeData
72      initialRoute: "/login",
73      routes: {
74        "/login": (_) => ChangeNotifierProvider(
75          create: (_) => LoginVM(),
76          child: const PantallaLogin(),
77        ), // ChangeNotifierProvider
78        "/registro": (_) => ChangeNotifierProvider(
79          create: (_) => RegistroVM(),
80          //create: (_) => RegistroVM(apiDni: api),
81          child: const PantallaRegistro(),
82        ), // ChangeNotifierProvider
83        "/registroAdmin": (_) => ChangeNotifierProvider(
84          create: (_) => AdminVM(apiDni: api),
85          child: const PantallaRegistroAdmin(),
86        ), // ChangeNotifierProvider
87        "/recuperar": (_) => ChangeNotifierProvider(
88          create: (_) => RecuperarVM(),
89          child: const PantallaRecuperar(),
90        ), // ChangeNotifierProvider
91        "/verificaEmail": (_) => const PantallaVerificaEmail(),
92        "/perfil": (_) => ChangeNotifierProvider(
93          create: (_) => PerfilVM(),
94          child: const PantallaPerfil(),
95        ), // ChangeNotifierProvider
96        "/inicio": (_) => AuthWrapper(
97          loginScreen: ChangeNotifierProvider(
98            create: (_) => LoginVM(),
99            child: const PantallaLogin(),
```

La parte:

- `"/inicioAdmin": (_) => const PantallaInicioAdmin(),`
- `"/adminUsuarios": (_) => const PantallaAdminUsuarios(),`
- `"/adminReportes": (_) => const PantallaAdminReporte(),`
- `"/adminNotificaciones": (_) => ChangeNotifierProvider(...)`

muestra que el patrón se repite:

- una ruta
- una pantalla
- o un provider + pantalla

```
100 .....), // ChangeNotifierProvider
101 .....homeScreenBuilder: (uid, userData) => ChangeNotifierProvider(
102 .....  create: (_) => NotificacionVM()..escucharNotificaciones(),
103 .....  child: const PantallaInicio(),
104 .....), // ChangeNotifierProvider
105 .....), // AuthWrapper
106 .....
107 .....  "/inicioAdmin": (_) => const PantallaInicioAdmin(),
108 .....  "/adminUsuarios": (_) => const PantallaAdminUsuarios(),
109 .....  "/adminReportes": (_) => const PantallaAdminReporte(),
110 .....  "/adminNotificaciones": (_) => ChangeNotifierProvider(
111 .....    create: (_) => AdminNotificacionesVM(),
112 .....    child: const PantallaAdminNotificaciones(),
113 .....  ), // ChangeNotifierProvider
```

La configuración de rutas y proveedores en la clase MyApp reutiliza el mismo patrón para distintos accesos, como login, registro y recuperación de contraseña, lo que incrementa la longitud del código y dificulta su mantenimiento.

En `auth_wrapper.dart`, el flujo de autenticación hace varias validaciones con patrones parecidos:

- verificar si existe usuario
- verificar si el documento existe
- verificar si los datos del usuario no son nulos
- verificar si la cuenta está activa
- cerrar sesión y devolver login

Es decir, el flujo se repite en varias ramas:

- si no hay usuario autenticado

- si el documento no existe
- si el usuarioData es null
- si la cuenta está desactivada

Cada caso hace algo parecido:

- imprimir un mensaje
- cerrar sesión
- volver al login

Eso sí es duplicación de lógica.

```
31 stream: FirebaseAuth.instance.authStateChanges(),
32 builder: (context, authSnapshot) {
33   // Si no hay usuario autenticado, mostrar login
34   if (!authSnapshot.hasData || authSnapshot.data == null) {
35     print('🔒 No hay usuario autenticado - Mostrando login');
36     return loginScreen;
37   }
38
39   final user = authSnapshot.data!;
40
41   // Escuchar cambios en el documento del usuario en tiempo real
42   return StreamBuilder<DocumentSnapshot>{
43     stream: FirebaseFirestore.instance
44       .collection('usuarios')
45       .doc(user.uid)
46       .snapshots(),
47     builder: (context, userSnapshot) {
48       // Mientras carga, mostrar indicador
49       if (userSnapshot.connectionState == ConnectionState.waiting) {
50         return const Scaffold(
51           backgroundColor: Colors.white,
52           body: Center(
53             child: Column(
54               mainAxisAlignment: MainAxisAlignment.center,
55               children: [
56                 CircularProgressIndicator(),
57                 SizedBox(height: 16),
58                 Text(
59                   'Verificando cuenta...',
60                   style: TextStyle(
```

```
59 ..... 'Verificando cuenta...',
60 ..... style: TextStyle(
61 .....   fontSize: 16,
62 .....   color: Colors.grey,
63 ..... ), // TextStyle
64 ..... ), // Text
65 ..... ],
66 ..... ), // Column
67 ..... ), // Center
68 ..... ); // Scaffold
69 ..... }
70 .....
71 ..... // ✗ Si hay error o no existe el documento
72 ..... if (userSnapshot.hasError ||
73 .....   !userSnapshot.hasData ||
74 .....   !userSnapshot.data!.exists) {
75 .....   print('⚠ Error o documento no existe para usuario: ${user.uid}');
76 .....   // Cerrar sesión y volver al login
77 .....   WidgetsBinding.instance.addPostFrameCallback((_) async {
78 .....     await FirebaseAuth.instance.signOut();
79 .....     if (context.mounted) {
80 .....       ScaffoldMessenger.of(context).showSnackBar(
81 .....         SnackBar(
82 .....           content: const Row(
83 .....             children: [
84 .....               Icon(Icons.error_outline, color: Colors.white),
85 .....               SizedBox(width: 12),
86 .....               Expanded(
87 .....                 child: Text(
88 .....                   'Error al cargar los datos del usuario.'
```

```
88 ..... Error al cargar los datos del usuario.',
89 ..... style: TextStyle(fontSize: 15),
90 ..... ), // Text
91 ..... ), // Expanded
92 ..... ],
93 ..... ), // Row
94 ..... backgroundColor: Colors.red,
95 ..... duration: const Duration(seconds: 4),
96 ..... behavior: SnackBarBehavior.floating,
97 ..... shape: RoundedRectangleBorder(
98 .....   borderRadius: BorderRadius.circular(10),
99 ..... ), // RoundedRectangleBorder
100 ..... ), // SnackBar
101 ..... );
102 ..... }
103 ..... });
104 ..... return loginScreen;
105 ..... }
106 .....
107 ..... // Obtener datos del usuario
108 ..... final userData = userSnapshot.data!.data() as Map<String, dynamic>;
109 .....
110 ..... if (userData == null) {
111 .....   print('⚠ userData es null para usuario: ${user.uid}');
112 .....   // Si no hay datos, cerrar sesión
113 .....   WidgetsBinding.instance.addPostFrameCallback(_) async {
114 .....     await FirebaseAuth.instance.signOut();
115 .....   };
116 .....   return loginScreen;
117 ..... }
```

```
119 .....// ✅ Verificar si la cuenta está activa
120 .....final bool isActive = userData['activo'] ?? true;
121 .....
122 .....print('👤 Usuario: ${user.email}');
123 .....print('📄 Estado activo: $isActive');
124 .....print('👤 Rol: ${userData['rol']}');
125 .....
126 .....// ❌ Si la cuenta está DESACTIVADA, cerrar sesión
127 .....if (!isActive) {
128 .....  print('❌ Cuenta DESACTIVADA - Cerrando sesión');
129 .....  WidgetsBinding.instance.addPostFrameCallback((_) async {
130 .....    await FirebaseAuth.instance.signOut();
131 .....    if (context.mounted) {
132 .....      ScaffoldMessenger.of(context).showSnackBar(
133 .....        SnackBar(
134 .....          content: const Row(
135 .....            children: [
136 .....              Icon(Icons.block, color: Colors.white, size: 28),
137 .....              SizedBox(width: 12),
138 .....              Expanded(
139 .....                child: Column(
140 .....                  crossAxisAlignment: CrossAxisAlignment.start,
141 .....                  mainAxisAlignment: MainAxisAlignment.min,
142 .....                  children: [
143 .....                    Text(
144 .....                      'Cuenta Desactivada',
145 .....                      style: TextStyle(
146 .....                        fontSize: 16,
147 .....                        fontWeight: FontWeight.bold,
```

```
142 ..... children: [
143 .....   Text(
144 .....     'Cuenta Desactivada',
145 .....     style: TextStyle(
146 .....       fontSize: 16,
147 .....       fontWeight: FontWeight.bold,
148 .....     ), // TextStyle
149 .....   ), // Text
150 .....   SizedBox(height: 4),
151 .....   Text(
152 .....     'Tu cuenta ha sido desactivada. Contacta al administrador para más i
153 .....     style: TextStyle(fontSize: 14),
154 .....   ), // Text
155 ..... ],
156 ..... ), // Column
157 ..... ), // Expanded
158 ..... ],
159 ..... ), // Row
160 ..... backgroundColor: Colors.red.shade700,
161 ..... duration: const Duration(seconds: 6),
162 ..... behavior: SnackBarBehavior.floating,
163 ..... shape: RoundedRectangleBorder(
164 .....   borderRadius: BorderRadius.circular(12),
165 ..... ), // RoundedRectangleBorder
166 ..... margin: const EdgeInsets.all(16),
167 ..... ), // SnackBar
168 ..... );
169 ..... }
170 ..... );
171 ..... return loginScreen;
```

Asimismo, en AuthWrapper se repite la lógica de validación de sesión, usuario activo y redirección al login en varios bloques. Esta duplicidad afecta la legibilidad del código y puede provocar inconsistencias si se modifica una validación sin actualizar el resto.

En `auth_wrapper.dart`

Los comentarios iniciales sí son útiles porque explican:

- qué hace `AuthWrapper`
- qué válida
- cuándo bloquea acceso

Eso es un comentario necesario.

```
1  import 'package:flutter/material.dart';
2  import 'package:firebase_auth/firebase_auth.dart';
3  import 'package:cloud_firestore/cloud_firestore.dart';
4
5  /// 🔵 AuthWrapper - Valida el estado de la cuenta en tiempo real
6  ///
7  /// Este wrapper monitorea constantemente:
8  /// - Si el usuario está autenticado
9  /// - Si la cuenta está activa (campo 'activo' en Firestore)
10 /// - Si el documento del usuario existe
11 ///
12 /// Si la cuenta es desactivada por un admin, el usuario será
13 /// expulsado automáticamente de la aplicación.
14 class AuthWrapper extends StatelessWidget {
15   /// Pantalla que se muestra cuando NO hay usuario autenticado
16   final Widget loginScreen;
17
18   /// Constructor que recibe el UID y los datos del usuario
19   /// para determinar qué pantalla mostrar
```

Pero también hay otros que son más repetitivos:

- “Si no hay usuario autenticado, mostrar login”
- “Escuchar cambios en el documento del usuario en tiempo real”
- “Mientras carga, mostrar indicador”
- “Si hay error o no existe el documento”
- “Verificar si la cuenta está activa”

```
28  @override
29  ✓  Widget build(BuildContext context) {
30  ✓      return StreamBuilder<User?>(
31          stream: FirebaseAuth.instance.authStateChanges(),
32  ✓      builder: (context, authSnapshot) {
33          // 🚫 Si no hay usuario autenticado, mostrar login
34  ✓      if (!authSnapshot.hasData || authSnapshot.data == null) {
35          print('🔒 No hay usuario autenticado - Mostrando login');
36          return loginScreen;
37      }
38  }
```

```
40
41      // 🕒 Escuchar cambios en el documento del usuario en tiempo real
42  ✓  return StreamBuilder<DocumentSnapshot>(
43      stream: FirebaseFirestore.instance
44          .collection('usuarios')
45          .doc(user.uid)
46          .snapshots(),
47  ✓  builder: (context, userSnapshot) {
48      // ⌚ Mientras carga, mostrar indicador
49      if (userSnapshot.connectionState == ConnectionState.waiting) {
50  ✓      return const Scaffold(
51          backgroundColor: Colors.white,
52  ✓      body: Center(
```

```
47      builder: (context, userSnapshot) {
48      // ⌚ Mientras carga, mostrar indicador
49      if (userSnapshot.connectionState == ConnectionState.waiting) {
50      return const Scaffold(
51          backgroundColor: Colors.white,
52      body: Center(
53          child: Column(
54              mainAxisAlignment: MainAxisAlignment.center,
55              children: [
56                  CircularProgressIndicator(),
57                  SizedBox(height: 16),
58                  Text(
59                      'Verificando cuenta...
```

```
70
71 // ✖ Si hay error o no existe el documento
72 if (userSnapshot.hasError ||
73     !userSnapshot.hasData ||
74     !userSnapshot.data!.exists) {
75     print('⚠ Error o documento no existe para usuario: ${user.uid}');
76     // Cerrar sesión y volver al login
77     WidgetsBinding.instance.addPostFrameCallback((_) async {
78         await FirebaseAuth.instance.signOut();
79         if (context.mounted) {
80             ScaffoldMessenger.of(context).showSnackBar(
81                 SnackBar(
82                     content: const Row(
83                         children: [
84                             Icon(Icons.error_outline, color: Colors.white),
85                             SizedBox(width: 12),
86                             Expanded(
87
```

```
118
119 // ✔ Verificar si la cuenta está activa
120 final bool isActive = userData['activo'] ?? true;
121
122 print('👤 Usuario: ${user.email}');
123 print('📄 Estado activo: $isActive');
124 print('👤 Rol: ${userData['rol']}');
125
126 // 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
127 if (!isActive) {
128     print('✖ Cuenta DESACTIVADA - Cerrando sesión');
129     WidgetsBinding.instance.addPostFrameCallback((_) async {
130         await FirebaseAuth.instance.signOut();
131         if (context.mounted) {
132             ScaffoldMessenger.of(context).showSnackBar(
133                 SnackBar(
```

Esto ya se entiende por el nombre de la variable y por el flujo del código, entonces esos comentarios no son indispensables.

- Fail Fast.

El código valida inmediatamente los casos críticos antes de permitir el acceso:

- si no hay usuario autenticado → devuelve la pantalla de login
- si el documento del usuario no existe → cierra sesión
- si los datos del usuario son nulos → vuelve al login
- si la cuenta está desactivada → cierra sesión y bloquea el acceso

```
27
28 @override
29 Widget build(BuildContext context) {
30   return StreamBuilder<User?>(
31     stream: FirebaseAuth.instance.authStateChanges(),
32     builder: (context, authSnapshot) {
33       // Si no hay usuario autenticado, mostrar login
34       if (!authSnapshot.hasData || authSnapshot.data == null) {
35         print('🔒 No hay usuario autenticado - Mostrando login');
36         return loginScreen;
37       }
38     }
39   );
40 }
```

```
70
71 // ❌ Si hay error o no existe el documento
72 if (userSnapshot.hasError ||
73     !userSnapshot.hasData ||
74     !userSnapshot.data!.exists) {
75   print('⚠️ Error o documento no existe para usuario: ${user.uid}');
76   // Cerrar sesión y volver al login
77   WidgetsBinding.instance.addPostFrameCallback((_) async {
```

```
106
107 // 📦 Obtener datos del usuario
108 final userData = userSnapshot.data!.data() as Map<String, dynamic>;
109
110 if (userData == null) {
111   print('⚠️ userData es null para usuario: ${user.uid}');
112   // Si no hay datos, cerrar sesión
113   WidgetsBinding.instance.addPostFrameCallback((_) async {
114     await FirebaseAuth.instance.signOut();
115   });
116   return loginScreen;
117 }
```

```
// Si la cuenta está DESACTIVADA, cerrar sesión
if (!isActive) {
  print('✗ Cuenta DESACTIVADA - Cerrando sesión');
  WidgetsBinding.instance.addPostFrameCallback((_) async {
    await FirebaseAuth.instance.signOut();
    if (context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: const Row(
            children: [
              Icon(Icons.block, color: Colors.white, size: 28),
              SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  mainAxisAlignment: MainAxisAlignment.min,
                  children: [
                    Text(
                      'Cuenta Desactivada',
                      style: TextStyle(
                        fontSize: 16,
                        fontWeight: FontWeight.bold,
                      ), // TextStyle
                    ), // Text
                    SizedBox(height: 4),
                    Text(
                      'Tu cuenta ha sido desactivada. Contacta al administrador para más información.',
                      style: TextStyle(fontSize: 14)
                    )
                  ],
                ),
              ),
            ],
          ),
        ),
      );
    }
  });
}
```

Eso es exactamente “Fail Fast”: detectar el problema lo más pronto posible y evitar continuar con un estado inválido.

- Evita los números mágicos.

Hay valores fijos que aparecen sin explicación clara, por ejemplo:

- rutas como `"/login"`, `"/perfil"`, `"/inicio"`
- colores como `0xFF4D9EF6` y `0xFFFF5F6FA`
- duración de mensajes como `Duration(seconds: 4)` o `Duration(seconds: 6)`
- tamaños fijos como 16, 28, 12, 10, etc.

Eso son números mágicos.

La regla dice que:

- si un valor tiene un significado importante, debe estar nombrado con una constante
- no dejarlo “flotando” en el código

```
46  +const MyApp({superKey});
47
48  @override
49  Widget build(BuildContext context) {
50    final api = ApiDniServicio(bearerToken: bearer);
51
52    return MaterialApp(
53      debugShowCheckedModeBanner: false,
54      title: 'SOS Mascota',
55      navigatorKey: navigatorKey,
56      theme: ThemeData(
57        useMaterial3: true,
58        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF4D9EF6)),
59        scaffoldBackgroundColor: const Color(0xFFFF5F6FA),
60        appBarTheme: const AppBarTheme(
61          backgroundColor: Colors.white,
62          foregroundColor: Colors.black,
63          elevation: 0.5,
64          centerTitle: true,
65          titleTextStyle: TextStyle(
66            fontSize: 18,
67            fontWeight: FontWeight.bold,
68            color: Colors.black,
69            // TextStyle
```

```
82 ..... content: const Row(  
83 .....   children: [  
84 .....     Icon(Icons.error_outline, color: Colors.white),  
85 .....     SizedBox(width: 12),  
86 .....     Expanded(  
87 .....       child: Text(  
88 .....         'Error al cargar los datos del usuario.',  
89 .....         style: TextStyle(fontSize: 15),  
90 .....       ), // Text  
91 .....     ), // Expanded  
92 .....   ],  
93 ..... ), // Row  
94 .....   backgroundColor: Colors.red,  
95 .....   duration: const Duration(seconds: 4),  
96 .....   behavior: SnackbarBehavior.floating,  
97 .....   shape: RoundedRectangleBorder(  
98 .....     borderRadius: BorderRadius.circular(10),  
99 .....   ), // RoundedRectangleBorder  
100 ..... ), // Snackbar  
101 ..... );  
102 ..... }  
103 ..... });  
104 ..... return loginScreen;  
105 ..... }  
106 .....  
107 ..... // Obtener datos del usuario  
108 ..... final userData = userSnapshot.data!.data() as Map<String, dynamic>;  
109 .....  
110 ..... if (userData == null) {  
111 .....   print('⚠️ userData es null para usuario: ${user.uid}');
```

“En la lógica de autenticación se detectan números mágicos en rutas, colores y tiempos de mensajes, lo que reduce la legibilidad del código y dificulta futuras modificaciones. Se recomienda reemplazarlos por constantes con nombres significativos.”

```
29   Widget build(BuildContext context) {
112   // Si no hay datos, cerrar sesión
113   WidgetsBinding.instance.addPostFrameCallback((_) async {
114     await FirebaseAuth.instance.signOut();
115   });
116   return loginScreen;
117 }
118
119 // ✅ Verificar si la cuenta está activa
120 final bool isActive = userData['activo'] ?? true;
121
122 print('👤 Usuario: ${user.email}');
123 print('📄 Estado activo: $isActive');
124 print('👤 Rol: ${userData['rol']}');
125
126 // 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
127 if (!isActive) {
128   print('❌ Cuenta DESACTIVADA - Cerrando sesión');
129   WidgetsBinding.instance.addPostFrameCallback((_) async {
130     await FirebaseAuth.instance.signOut();
131     if (context.mounted) {
132       ScaffoldMessenger.of(context).showSnackBar(
133         SnackBar(
134           content: const Row(
135             children: [
136               Icon(Icons.block, color: Colors.white, size: 28),
137               SizedBox(width: 12),
138               Expanded(
139                 child: Column(
140                   crossAxisAlignment: CrossAxisAlignment.start,
141                   mainAxisAlignment: MainAxisAlignment.min
```

```
141 ..... mainAxisSize: MainAxisSize.min,  
142 ..... children: [  
143 .....   Text(  
144 .....     'Cuenta Desactivada',  
145 .....     style: TextStyle(  
146 .....       fontSize: 16,  
147 .....       fontWeight: FontWeight.bold,  
148 .....     ), // TextStyle  
149 .....   ), // Text  
150 .....   SizedBox(height: 4),  
151 .....   Text(  
152 .....     'Tu cuenta ha sido desactivada. Contacta al administrador',  
153 .....     style: TextStyle(fontSize: 14),  
154 .....   ), // Text  
155 ..... ],  
156 ..... ), // Column  
157 ..... ), // Expanded  
158 ..... ],  
159 ..... ), // Row  
160 ..... backgroundColor: Colors.red.shade700,  
161 ..... duration: const Duration(seconds: 6),  
162 ..... behavior: SnackBarBehavior.floating,  
163 ..... shape: RoundedRectangleBorder(  
164 .....   borderRadius: BorderRadius.circular(12),  
165 ..... ), // RoundedRectangleBorder  
166 ..... margin: const EdgeInsets.all(16),  
167 ..... ), // SnackBar
```



- AuthWrapper
- loginScreen
- homeScreenBuilder
- userData

```
13 // expulsado automáticamente de la aplicación.
14 class AuthWrapper extends StatelessWidget {
15   /// Pantalla que se muestra cuando NO hay usuario autenticado
16   final Widget loginScreen;
17
18   /// Constructor que recibe el UID y los datos del usuario
19   /// para determinar qué pantalla mostrar
20   final Widget Function(String uid, Map<String, dynamic> userData) homeScreenBuilder;
21
22   const AuthWrapper({
23     super.key,
24     required this.loginScreen,
```

- bearer
- navigatorKey

- bearer no dice qué tipo de token es ni para qué sirve
- navigatorKey es un nombre aceptable, pero no es tan descriptivo como appNavigatorKey

```
import 'servicios/api_dni_servicio.dart';  
  
final String bearer =
```

- 2 -

- authSnapshot
- userSnapshot
- userData
- isActive

```
27
28   @override
29   Widget build(BuildContext context) {
30     return StreamBuilder<User?>(
31       stream: FirebaseAuth.instance.authStateChanges(),
32       builder: (context, authSnapshot) {
33         // Si no hay usuario autenticado, mostrar login
34         if (!authSnapshot.hasData || authSnapshot.data == null) {
35           print('🔒 No hay usuario autenticado - Mostrando login');
36           return loginScreen;
37         }
38
39         final user = authSnapshot.data!;
```

```
46     .snapshots(),
47     builder: (context, userSnapshot) {
48         // ⌚ Mientras carga, mostrar indicador
49         if (userSnapshot.connectionState == ConnectionState.waiting)
50             return const Scaffold(
51                 backgroundColor: Colors.white,
52                 body: Center(
53                     child: Column(
54                         mainAxisAlignment: MainAxisAlignment.center,
55                         children: [
56                             CircularProgressIndicator(),
57                             SizedBox(height: 16),
58                             Text(
59                                 'Verificando cuenta...',
60                                 style: TextStyle(
61                                     fontSize: 16,
62                                     color: Colors.grey,
63                                 ), // TextStyle
64                             ), // Text
65                         ],
66                     ), // Column
67                 ), // Center
68             ); // Scaffold
69     }
70
71     // ✖ Si hay error o no existe el documento
72     if (userSnapshot.hasError ||
73         !userSnapshot.hasData ||
74         !userSnapshot.data!.exists) {
```

```
        // 📦 Obtener datos del usuario
        final userData = userSnapshot.data!.data() as Map<String, dynamic>?;
        if (userData == null) {
            print('⚠ userData es null para usuario: ${user.uid}');
            // Si no hay datos, cerrar sesión
            WidgetsBinding.instance.addPostFrameCallback((_) async {
                await FirebaseAuth.instance.signOut();
            });
            return loginScreen;
        }
    }
```


- Devuelve valores, no los imprimas.

En auth_wrapper.dart: hay varios print() y debugPrint dentro de la lógica de autenticación:

- print('🔒 No hay usuario autenticado - Mostrando login');
- print('⚠️ Error o documento no existe para usuario: \${user.uid}');
- print('⚠️ userData es null para usuario: \${user.uid}');
- print('🔍 Usuario: \${user.email}');
- print('❌ Cuenta DESACTIVADA - Cerrando sesión');
- print('✅ Cuenta ACTIVA - Permitiendo acceso');

En mi caso, el flujo de autenticación:

- decide si mostrar login
- decide si cerrar sesión
- decide si permitir acceso

Pero en lugar de devolver un estado claro, se están mandando mensajes por consola.

```
29 widget Build(BuildContext context) {
30   return StreamBuilder<User?>(<
31     stream: FirebaseAuth.instance.authStateChanges(),
32     builder: (context, authSnapshot) {
33       // ⚠️ Si no hay usuario autenticado, mostrar login
34       if (!authSnapshot.hasData || authSnapshot.data == null) {
35         print('🔒 No hay usuario autenticado - Mostrando login');
36         return loginScreen;
37       }
38
39       // ⚠️ Error o documento no existe para usuario: ${user.uid}
40       if (!userSnapshot.hasData || !userSnapshot.data!.exists) {
41         print('⚠️ Error o documento no existe para usuario: ${user.uid}');
42         // Cerrar sesión y volver al login
43         WidgetsBinding.instance.addPostFrameCallback((_) async {
44           await FirebaseAuth.instance.signOut();
45           if (context.mounted) {
46             ScaffoldMessenger.of(context).showSnackBar(
```

```
109
110     if (userData == null) {
111         print('⚠ userData es null para usuario: ${user.uid}');
112         // Si no hay datos, cerrar sesión
113         WidgetsBinding.instance.addPostFrameCallback((_) async {
114             await FirebaseAuth.instance.signOut();
115         });
116         return loginScreen;
117     }
118
119     // 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
120     if (!isActive) {
121         print('❌ Cuenta DESACTIVADA - Cerrando sesión');
122         WidgetsBinding.instance.addPostFrameCallback((_) async {
123             await FirebaseAuth.instance.signOut();
124             if (context.mounted) {
125                 ScaffoldMessenger.of(context).showSnackBar(
126
127
128
129
130
131
132
133
134
135
136
137
138
139     // ✅ Verificar si la cuenta está activa
140     final bool isActive = userData['activo'] ?? true;
141
142     print('🔍 Usuario: ${user.email}');
143     print('📄 Estado activo: $isActive');
144     print('👤 Rol: ${userData['rol']}');
```

Donde se usan print() para informar el estado de la autenticación en lugar de devolver un resultado o manejarlo como valor.

- Utiliza los espacios en blanco para mejor lectura.

Se ve una buena utilización de espacios y sangría en auth_wrapper.dart.

Qué está bien

Hay:

- indentación correcta
- líneas separadas lógicamente
- bloques bien organizados
- espacios entre estructuras y dentro de widgets

Por ejemplo:

- if (...) {
- return const Scaffold(...)
- children: [...]
- content: const Row(...)

Todo eso hace que el código sea más legible.

```
// Si no hay usuario autenticado, mostrar login
if (!authSnapshot.hasData || authSnapshot.data == null) {
  print('❗ No hay usuario autenticado - Mostrando login');
  return loginScreen;
}

final user = authSnapshot.data!;

// 🕒 Escuchar cambios en el documento del usuario en tiempo real
return StreamBuilder<DocumentSnapshot>(
  stream: FirebaseFirestore.instance
    .collection('usuarios')
    .doc(user.uid)
    .snapshots(),
  builder: (context, userSnapshot) {
    // ⌚ Mientras carga, mostrar indicador
    if (userSnapshot.connectionState == ConnectionState.waiting) {
      return const Scaffold(
        backgroundColor: Colors.white,
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const SizedBox(height: 20),
              const CircularProgressIndicator(),
              const Text('Cargando...'),
            ],
          ),
        ),
      );
    }
    // 🏠 Después de cargar, mostrar pantalla de inicio
    return homeScreen();
  },
);
```

```

130 ..... await FirebaseAuth.instance.signOut();
131 ..... if (context.mounted) {
132 .....   ScaffoldMessenger.of(context).showSnackBar(
133 .....     SnackBar(
134 .....       content: const Row(
135 .....         children: [
136 .....           Icon(Icons.block, color: Colors.white, size: 24),
137 .....           SizedBox(width: 12),
138 .....           Expanded(
139 .....             child: Column(
140 .....               crossAxisAlignment: CrossAxisAlignment.start,
141 .....               mainAxisAlignment: MainAxisAlignment.min,
142 .....               children: [
143 .....                 Text(
144 .....                   'Cuenta Desactivada',
145 .....                   style: TextStyle(
146 .....                     fontSize: 16,
147 .....                     fontWeight: FontWeight.bold,
148 .....                   ), // TextStyle
149 .....                 ), // Text
150 .....                 SizedBox(height: 4),
151 .....                 Text(
152 .....                   'Tu cuenta ha sido desactivada. Contacta
153 .....                   style: TextStyle(fontSize: 14),

```

```

134         content: const Row(
135           children: [
136             Icon(Icons.block, color: Colors.white, size:
137               SizedBox(width: 12),
138             Expanded(
139               child: Column(
140                 crossAxisAlignment: CrossAxisAlignment.start,
141                 mainAxisAlignment: MainAxisAlignment.min,
142                 children: [
143                   Text(
144                     'Cuenta Desactivada',
145                     style: TextStyle(
146                       fontSize: 16,
147                       fontWeight: FontWeight.bold,
148                     ), // TextStyle
149                   ), // Text
150                   SizedBox(height: 4),
151                   Text(
152                     'Tu cuenta ha sido desactivada. Contacta
153                     style: TextStyle(fontSize: 14),
154                   ), // Text
155                 ],

```

También en app.dart hay muchas rutas seguidas sin una separación más clara de módulos o grupos de navegación.

```
initialRoute: "/login",
routes: {
  "/login": (_) => ChangeNotifierProvider(
    create: (_) => LoginVM(),
    child: const PantallaLogin(),
  ), // ChangeNotifierProvider
  "/registro": (_) => ChangeNotifierProvider(
    create: (_) => RegistroVM(),
    //create: (_) => RegistroVM(apiDni: api),
    child: const PantallaRegistro(),
  ), // ChangeNotifierProvider
  "/registroAdmin": (_) => ChangeNotifierProvider(
    create: (_) => AdminVM(apiDni: api),
    child: const PantallaRegistroAdmin(),
  ), // ChangeNotifierProvider
  "/recuperar": (_) => ChangeNotifierProvider(
    create: (_) => RecuperarVM(),
    child: const PantallaRecuperar(),
  ), // ChangeNotifierProvider
  "/verificaEmail": (_) => const PantallaVerificaEmail(),
  "/perfil": (_) => ChangeNotifierProvider(
    create: (_) => PerfilVM(),
    child: const PantallaPerfil(),
  ), // ChangeNotifierProvider
  "/inicio": (_) => AuthWrapper(
    loginScreen: ChangeNotifierProvider(
      create: (_) => LoginVM(),
      child: const PantallaLogin(),
    ), // ChangeNotifierProvider
```

```
95         ), // ChangeNotifierProvider
96     "/inicio": (_) => AuthWrapper(
97         loginScreen: ChangeNotifierProvider(
98             create: (_) => LoginVM(),
99             child: const PantallaLogin(),
100         ), // ChangeNotifierProvider
101     homeScreenBuilder: (uid, userData) => ChangeNotifierProvider(
102         create: (_) => NotificacionVM()..escucharNotificaciones(),
103         child: const PantallaInicio(),
104     ), // ChangeNotifierProvider
105     ), // AuthWrapper
106
107     "/inicioAdmin": (_) => const PantallaInicioAdmin(),
108     "/adminUsuarios": (_) => const PantallaAdminUsuarios(),
109     "/adminReportes": (_) => const PantallaAdminReporte(),
110     "/adminNotificaciones": (_) => ChangeNotifierProvider(
111         create: (_) => AdminNotificacionesVM(),
112         child: const PantallaAdminNotificaciones(),
113     ), // ChangeNotifierProvider
```



Código mejorado:

En app.dart:

No te repitas

Se centralizó la configuración con AppConfig y las rutas con AppRoutes, evitando repetir el mismo patrón de providers y pantallas en cada ruta.

Comenta lo necesario

Se dejó solo la documentación útil y clara para la configuración y la validación del flujo.

Fail Fast

La app ahora organiza mejor el arranque y la navegación desde una estructura central, reduciendo el riesgo de cargar rutas con configuración inválida.

Evita los números mágicos

Se sacaron valores como colores, tiempos y tamaños a app_config.dart, evitando que queden dispersos por el código.

Utiliza buenos nombres

Se usaron nombres descriptivos como:

- AppConfig
- AppRoutes
- login
- registro
- inicioAdmin

No uses variables globales

Se movió la configuración global a una clase centralizada, evitando que esos valores estén repartidos por todo el archivo.

Devuelve valores, no los imprimas

La lógica principal ya no depende de print() para decidir el flujo; se usa validación directa y retorno de pantallas.

Utiliza los espacios en blanco para mejor lectura

La separación entre configuración, rutas y widget principal mejora mucho la legibilidad del archivo.

```
40  /// Nombres de ruta centralizados como constantes.
41  /// Evita escribir el mismo string ("/login", "/perfil", etc.) suelto
42  /// en distintas partes del proyecto (strings "mágicos").
43  class AppRoutes {
44      static const String login = '/login';
45      static const String registro = '/registro';
46      static const String registroAdmin = '/registroAdmin';
47      static const String recuperar = '/recuperar';
48      static const String verificaEmail = '/verificaEmail';
49      static const String perfil = '/perfil';
50      static const String inicio = '/inicio';
51      static const String inicioAdmin = '/inicioAdmin';
52      static const String adminUsuarios = '/adminUsuarios';
53      static const String adminReportes = '/adminReportes';
54      static const String adminNotificaciones = '/adminNotificaciones';
55      static const String reportarMascota = '/reportarMascota';
56      static const String avistamiento = '/avistamiento';
57      static const String verReportes = '/verReportes';
58      static const String misReportes = '/misReportes';
59      static const String notificaciones = '/notificaciones';
60      static const String comentarios = '/comentarios';
61      static const String mapa = '/mapa';
62
63      static Map<String, WidgetBuilder> build() {
64          final api = ApiDniServicio(bearerToken: AppConfig.bearerToken);
65
66          return {
67              login: (_) => _withProvider(LoginVM.new, const PantallaLogin()),
68              registro: (_) => _withProvider(RegistroVM.new, const PantallaRegistro()),
69              registroAdmin: (_) =>
```

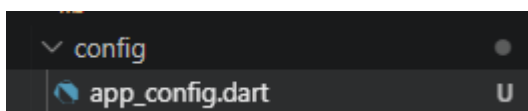
```
69     registroAdmin: (_) =>  
70     | _withProvider(() => AdminVM(apiDni: api), const PantallaRegistroAdmin()),  
71     recuperar: (_) => _withProvider(RecuperarVM.new, const PantallaRecuperar()),  
72     verificaEmail: (_) => const PantallaVerificaEmail(),  
73     perfil: (_) => _withProvider(PerfilVM.new, const PantallaPerfil()),  
74  
75     inicio: (_) => AuthWrapper(  
76     | loginScreen: _withProvider(LoginVM.new, const PantallaLogin()),  
77     | homeScreenBuilder: (uid, userData) =>  
78     | | _withNotificaciones(const PantallaInicio()),  
79     | ), // AuthWrapper  
80  
81     inicioAdmin: (_) => const PantallaInicioAdmin(),  
82     adminUsuarios: (_) => const PantallaAdminUsuarios(),  
83     adminReportes: (_) => const PantallaAdminReporte(),  
84     adminNotificaciones: (_) => _withProvider(  
85     | AdminNotificacionesVM.new,  
86     | const PantallaAdminNotificaciones(),  
87     | ),  
88  
89     reportarMascota: (_) => const PantallaReporteMascota(),  
90     avistamiento: (_) => const PantallaAvistamiento(),  
91     verReportes: (_) => const PantallaVerReportes(),  
92     misReportes: (_) => const PantallaMisReportes(),  
93  
94     notificaciones: (_) => _withNotificaciones(const PantallaNotificaciones()),  
95     comentarios: (_) => const PantallaComentarios(),  
96     mapa: (_) => const PantallaMapaInteractivo(),  
97 };
```

```
99  
100 /// Envuelve [child] con el `ChangeNotifierProvider` de [create].  
101 /// Reemplaza el bloque `ChangeNotifierProvider(create: ..., child: ...)`  
102 /// que antes se repetía manualmente para cada pantalla con ViewModel.  
103 static Widget _withProvider<T extends ChangeNotifier>(  
104     T Function() create,  
105     Widget child,  
106 ) {  
107     return ChangeNotifierProvider<T>(create: (_) => create(), child: child);  
108 }  
109  
110 /// `NotificacionVM` se inicializaba igual en "/inicio" y "/notificaciones".  
111 /// Se centraliza aquí para no repetir la misma lógica dos veces.  
112 static Widget _withNotificaciones(Widget child) {  
113     return ChangeNotifierProvider(  
114     | create: (_) => NotificacionVM()..escucharNotificaciones(),  
115     | child: child,  
116     | ); // ChangeNotifierProvider  
117 }  
118 }
```

```
119
120 class MyApp extends StatelessWidget {
121   const MyApp({super.key});
122
123   @override
124   Widget build(BuildContext context) {
125     return MaterialApp(
126       debugShowCheckedModeBanner: false,
127       title: AppConfig.appTitle,
128       theme: ThemeData(
129         useMaterial3: true,
130         colorScheme: ColorScheme.fromSeed(seedColor: ■ AppConfig.primaryColor),
131         scaffoldBackgroundColor: ■ AppConfig.scaffoldBackground,
132         appBarTheme: const AppBarTheme(
133           backgroundColor: ■ Colors.white,
134           foregroundColor: □ Colors.black,
135           elevation: 0.5,
136           centerTitle: true,
137           titleTextStyle: TextStyle(
138             fontSize: 18,
139             fontWeight: FontWeight.bold,
140             color: □ Colors.black,
141           ), // TextStyle
142         ), // AppBarTheme
143       ), // ThemeData
144       initialRoute: AppRoutes.login,
145       routes: AppRoutes.build(),
146     ); // MaterialApp
147   }
148 }
```

Se creó una ruta en config/app_config.dart.

```
5 import 'package:sos_mascotas/vista/usuario/pantalla_notificacion.dart';
6 import 'package:sos_mascotas/vistamodelo/notificacion/notificacion_vm.dart';
7 import 'config/app_config.dart';
8 import 'utils/auth_wrapper.dart';
9
```



```
1  /// Configuración global de la app
2  ///
3  /// Centraliza valores que antes estaban repetidos como "números mágicos"
4  /// (colores, duraciones, tamaños de SnackBar, etc.) para que un cambio
5  /// de diseño no obligue a tocar varios archivos. `auth_wrapper.dart`
6  /// importa esta clase desde aquí (import circular permitido en Dart).
7  import 'package:flutter/material.dart';
8
9  class AppConfig {
10   AppConfig._(); // Evita instanciar esta clase; solo se usa de forma estática.
11
12   static const String appTitle = 'SOS Mascota';
13
14   static const String bearerToken = String.fromEnvironment(
15     'BEARER_TOKEN',
16     defaultValue: '',
17   );
18
19   // Colores base del tema
20   static const Color primaryColor = Color(0xFF4D9EF6);
21   static const Color scaffoldBackground = Color(0xFFF5F6FA);
22   // Duraciones de SnackBar
23   static const Duration snackDurationDefault = Duration(seconds: 4);
24   static const Duration snackDurationLong = Duration(seconds: 6);
25   // Tamaños y espaciados usados en los avisos (SnackBars) de AuthWrapper
26   static const double snackIconSize = 28;
27   static const double snackSpacing = 12;
28   static const double snackTextSpacing = 4;
29   static const double snackBorderRadiusDefault = 10;
30   static const double snackBorderRadiusLarge = 12;
31   static const double snackMargin = 16;
32 }
```

En auth_wrapper.dart.

**No te repitas**

Se unificó la lógica de cierre de sesión y mensaje de error para evitar duplicar el mismo proceso en varios casos.

Comenta lo necesario

Se dejó solo la documentación clave sobre lo que hace AuthWrapper y cómo valida la sesión.

Fail Fast

Se valida inmediatamente:

- si no hay sesión
- si el documento del usuario no existe
- si el usuario es nulo
- si la cuenta está desactivada

Si algo falla, devuelve al login de inmediato.

Evita los números mágicos

Se usaron constantes de app_config.dart para:

- tiempo del SnackBar
- tamaños
- radios
- espaciados

Utiliza buenos nombres

Los nombres AuthWrapper, homeScreenBuilder, cuentaActiva y docExiste son claros y descriptivos.

No uses variables globales

La validación se hace local a la sesión y al documento, sin depender de valores globales a lo largo de la app.

Devuelve valores, no los imprimas

Se eliminó la lógica de depuración por consola y se reemplazó por decisiones de flujo y mensajes visibles para el usuario.

Utiliza los espacios en blanco para mejor lectura

La estructura del StreamBuilder, validaciones y retorno está bien separada en bloques.

```
1  import 'package:flutter/material.dart';
2  import 'package:firebase_auth/firebase_auth.dart';
3  import 'package:cloud_firestore/cloud_firestore.dart';
4
5  import '../config/app_config.dart';
6
7  /// Verifica en tiempo real que el usuario esté autenticado, que su
8  /// documento exista en Firestore y que su cuenta siga activa.
9  ///
10 /// Aplica "fail fast": ante cualquier condición inválida (sin sesión,
11 /// documento inexistente, error de lectura o cuenta desactivada) cierra
12 /// sesión de inmediato y regresa a `loginScreen`, en vez de dejar que
13 /// la app siga en un estado a medias.
14 class AuthWrapper extends StatelessWidget {
15   final Widget loginScreen;
16   final Widget Function(String uid, Map<String, dynamic> userData) homeScreenBuilder;
17
18   const AuthWrapper({
19     super.key,
20     required this.loginScreen,
21     required this.homeScreenBuilder,
22   });
23
24   @override
25   Widget build(BuildContext context) {
26     return StreamBuilder<User?>(
27       stream: FirebaseAuth.instance.authStateChanges(),
28       builder: (context, authSnapshot) {
29         final user = authSnapshot.data;
```

```
24 @override
25 Widget build(BuildContext context) {
26   return StreamBuilder<User?>(
27     stream: FirebaseAuth.instance.authStateChanges(),
28     builder: (context, authSnapshot) {
29       final user = authSnapshot.data;
30       if (user == null) return loginScreen; // Fail fast: sin sesión, no hay nada más que revisar
31
32       return StreamBuilder<DocumentSnapshot>(
33         stream: FirebaseFirestore.instance
34           .collection('usuarios')
35           .doc(user.uid)
36           .snapshots(),
37         builder: (context, userSnapshot) {
38           if (userSnapshot.connectionState == ConnectionState.waiting) {
39             return const _VerificandoCuenta();
40           }
41
42           final docExiste = userSnapshot.hasData && userSnapshot.data!.exists;
43           if (userSnapshot.hasError || !docExiste) {
44             _cerrarSesionYAvisar(
45               context,
46               mensaje: 'Error al cargar los datos del usuario.',
47               icono: Icons.error_outline,
48               color: Colors.red,
49               duracion: AppConfig.snackDurationDefault,
50             );
51             return loginScreen;
52           }
53         }
54       );
55     }
56   );
57 }
```

```
53
54 final userData = userSnapshot.data!.data() as Map<String, dynamic>;
55 if (userData == null) {
56   FirebaseAuth.instance.signOut();
57   return loginScreen;
58 }
59
60 final bool cuentaActiva = userData['activo'] ?? true;
61 if (!cuentaActiva) {
62   _cerrarSesionYAvisar(
63     context,
64     titulo: 'Cuenta Desactivada',
65     mensaje:
66       'Tu cuenta ha sido desactivada. Contacta al administrador para más información.',
67     icono: Icons.block,
68     color: Colors.red.shade700,
69     duracion: AppConfig.snackDurationLong,
70   );
71   return loginScreen;
72 }
73
74 return homeScreenBuilder(user.uid, userData);
75 },
76 ); // StreamBuilder
77 },
78 ); // StreamBuilder
79 }
```

```
80
81    /// Cierra sesión y muestra un SnackBar con el motivo.
82    /// Antes este mismo patrón (signOut + SnackBar) estaba duplicado
83    /// para el caso de "error al cargar datos" y el de "cuenta desactivada".
84    void _cerrarSesionYAvisar(
85        BuildContext context, {
86        String? titulo,
87        required String mensaje,
88        required IconData icono,
89        required Color color,
90        required Duration duracion,
91    }) {
92        WidgetsBinding.instance.addPostFrameCallback(_) async {
93            await FirebaseAuth.instance.signOut();
94            if (!context.mounted) return;
95
96            final esAvisoImportante = titulo != null;
97
98            ScaffoldMessenger.of(context).showSnackBar(
99                SnackBar(
100                    content: Row(
101                        children: [
102                            Icon(icono, color: Colors.white, size: AppConfig.snackIconSize),
103                            SizedBox(width: AppConfig.snackSpacing),
104                            Expanded(
105                                child: Column(
106                                    crossAxisAlignment: CrossAxisAlignment.start,
107                                    mainAxisAlignment: MainAxisAlignment.min,
108                                    children: [
109                                        if (esAvisoImportante) ...[
110                                            Text(
```

```
131         ), // RoundedRectangleBorder
132         margin: esAvisoImportante ? const EdgeInsets.all(AppConfig.snackMargin) : null,
133       ), // SnackBar
134     );
135   });
136 }
137 }
138
139 /// Pantalla de carga mientras se valida el estado de la cuenta contra Firestore.
140 class _VerificandoCuenta extends StatelessWidget {
141   const _VerificandoCuenta();
142
143   @override
144   Widget build(BuildContext context) {
145     return const Scaffold(
146       backgroundColor: Colors.white,
147       body: Center(
148         child: Column(
149           mainAxisAlignment: MainAxisAlignment.center,
150           children: [
151             CircularProgressIndicator(),
152             SizedBox(height: 16),
153             Text('Verificando cuenta...', style: TextStyle(fontSize: 16, color: Colors.grey)),
154           ],
155         ), // Column
156       ), // Center
157     ); // Scaffold
158   }
159 }
```

Se corrigió la autenticación de usuario validando la sesión en Firebase Auth, la existencia del documento del usuario en Firestore y el estado activo de la cuenta. Además, se redirige al login cuando la cuenta no cumple con los requisitos de acceso. En app.dart se reorganizó la navegación y la configuración para reducir la repetición y mejorar la mantenibilidad.

Autenticar Usuario se realizó en estos 2 archivos por:

- auth_wrapper.dart: validar sesión, validar identidad, permitir o bloquear acceso, controlar usuarios activos/inactivos.
- app.dart : ayuda a organizar la navegación y configuración.

- **Registrar reporte de mascota**

En pantalla_reporte_mascota.dart

- No te repitas.

El bloque completo de "seleccionar imagen → mostrar snackbar de análisis → subir foto → mostrar snackbar de éxito/error" está copiado 3 veces casi idéntico:

en el onTap de la imagen principal (ImageSource.gallery)

```
223 onTap: () async {
224   final picker = ImagePicker();
225   final picked = await picker.pickImage(
226     source: ImageSource.gallery,
227   );
228
229   if (picked != null) {
230     try {
231       ScaffoldMessenger.of(context).showSnackBar(
232         const SnackBar(
233           content: Text("Analizando imagen... 🟡"),
234           backgroundColor: Colors.blueAccent,
235           duration: Duration(seconds: 2),
236         ), // SnackBar
237       );
238
239       dynamic imagenParaEnviar;
240
241       if (kIsWeb) {
242         // En Web -> usamos bytes
243         imagenParaEnviar = await picked.readAsBytes();
244       } else {
245         // En Android/iOS -> usamos File normal
246         imagenParaEnviar = File(picked.path);
247       }
248
249       final url = await vm.subirFoto(imagenParaEnviar);
250       vm.agregarFoto(url);
251     } catch (e) {
252       // Manejar error
253     }
254   }
255 }
```

```
252 ScaffoldMessenger.of(context).showSnackBar(  
253   const SnackBar(  
254     content: Text(  
255       "✅ Imagen válida detectada y subida correctamente",  
256     ), // Text  
257     backgroundColor: Colors.green,  
258   ), // SnackBar  
259 );  
260 } catch (e) {  
261   ScaffoldMessenger.of(context).showSnackBar(  
262     SnackBar(  
263       content: Text(  
264         e.toString().replaceAll("Exception: ", ""),  
265         style: const TextStyle(  
266           fontWeight: FontWeight.bold,  
267         ), // TextStyle  
268       ), // Text  
269       backgroundColor: Colors.redAccent,  
270     ), // SnackBar  
271   );  
272 }  
273 }  
274 },
```

```
386 // ... (onPressed y estilo)
387 onPressed: () async {
388   final picker = ImagePicker();
389   final picked = await picker.pickImage(
390     source: ImageSource.camera,
391   );
392   if (picked == null) return;
393
394   try {
395     ScaffoldMessenger.of(context).showSnackBar(
396       const SnackBar(
397         content: Text("Analizando imagen... 📷"),
398         backgroundColor: Colors.blueAccent,
399         duration: Duration(seconds: 2),
400       ), // SnackBar
401     );
402
403     dynamic imagenParaVM;
404
405     if (kIsWeb) {
406       // En Web -> usamos bytes del archivo
407       imagenParaVM = await picked.readAsBytes();
408     } else {
409       // En Android/iOS -> usamos File normal
410       imagenParaVM = File(picked.path);
411     }
412
413     final url = await vm.subirFoto(imagenParaVM);
414
```

```
415 vm.agregarFoto(url);
416
417 ScaffoldMessenger.of(context).showSnackBar(
418   const SnackBar(
419     content: Text(
420       "✅ Imagen válida detectada y subida correctamente",
421     ), // Text
422     backgroundColor: Colors.green,
423   ), // SnackBar
424 );
425 } catch (e) {
426   ScaffoldMessenger.of(context).showSnackBar(
427     SnackBar(
428       content: Text(
429         e.toString().replaceAll("Exception: ", ""),
430         style: const TextStyle(
431           fontWeight: FontWeight.bold,
432         ), // TextStyle
433       ), // Text
434       backgroundColor: Colors.redAccent,
435     ), // SnackBar
436   );
437 }
438 },
```

En el botón "Galería" (ImageSource.gallery otra vez)

```
457 // ... (onPressed y estilo)
458 onPressed: () async {
459   final picker = ImagePicker();
460   final picked = await picker.pickImage(
461     source: ImageSource.gallery,
462   );
463   if (picked == null) return;
464
465   try {
466     ScaffoldMessenger.of(context).showSnackBar(
467       const SnackBar(
468         content: Text("Analizando imagen... 🌀"),
469         backgroundColor: Colors.blueAccent,
470         duration: Duration(seconds: 2),
471       ), // SnackBar
472     );
473
474     dynamic imagenParaVM;
475
476     if (kIsWeb) {
477       // 🌐 WEB: el picker devuelve bytes, NO File
478       imagenParaVM = await picked.readAsBytes();
479     } else {
480       // 📱 MOVIL: el picker devuelve un path válido
481       imagenParaVM = File(picked.path);
482     }
483
484     final url = await vm.subirFoto(imagenParaVM);
485     vm.agregarFoto(url);
```

```
487 ScaffoldMessenger.of(context).showSnackBar(  
488   const SnackBar(  
489     content: Text(  
490       "✅ Imagen válida detectada y subida correctamente",  
491     ), // Text  
492     backgroundColor: Colors.green,  
493   ), // SnackBar  
494 );  
495 } catch (e) {  
496   ScaffoldMessenger.of(context).showSnackBar(  
497     SnackBar(  
498       content: Text(  
499         e.toString().replaceAll("Exception: ", ""),  
500       style: const TextStyle(  
501         fontWeight: FontWeight.bold,  
502       ), // TextStyle  
503     ), // Text  
504     backgroundColor: Colors.redAccent,  
505   ), // SnackBar  
506 );  
507 }  
508 },
```

Los tres hacen exactamente lo mismo (seleccionar imagen → SnackBar "Analizando imagen..." → subir → SnackBar de éxito/error), solo cambia el ImageSource.

También se repite la lógica de "S/." al mostrar el monto (la función anónima `() { ... }()`):
Si el monto se formatea distinto en más de un lugar, terminará duplicándose también.

```
943         if (r.montoRecompensa.isNotEmpty) ...[
944             (() {
945                 String m = r.montoRecompensa.trim();
946                 if (!m.startsWith(RegExp(r'^S/\.?\\s*'))) {
947                     m = 'S/. $m';
948                 }
949                 return _infoRow("💰 Monto", m);
```

- Comenta lo necesario.

Hay comentarios que sí ayudan (ej. explicar por qué se usa `.skip(1)` para las miniaturas).

Pero hay varios que sobran o son señales de código incompleto:

- Definiciones auxiliares (`_sectionTitle`, `_card`, etc.) no incluidas aquí por espacio, pero son necesarias para que el código compile. esto es una nota de "borrador", no debería quedar en el código final.

```
95     ) async {
96         // Definiciones auxiliares (_sectionTitle, _card, etc.) no incluidas aquí por espacio,
97         // pero son necesarias para que el código compile.
98
99         return showDialog(
100             context: context,
101             builder: (BuildContext context) {
102                 return AlertDialog(
103                     title: const Text("Eliminar foto"),
```

- `// ... (onPressed y estilo)` y `// ... (Icono de cámara y textos)` : comentarios placeholder que no aportan nada y quedaron de cuando se recortó el código.

```
386         // ... (onPressed y estilo)
387         onPressed: () async {
388             final picker = ImagePicker();
389             final picked = await picker.pickImage(
390                 source: ImageSource.camera,
391             );
392             if (picked == null) return;
393
```

```

448         padding: const EdgeInsets.symmetric(
449             horizontal: 16,
450             vertical: 10,
451         ), // EdgeInsets.symmetric
452     ),
453 ), // ElevatedButton.icon
454 const SizedBox(width: 12),
455 // Galería (foto)
456 ElevatedButton.icon(
457     // ... (onPressed y estilo)

```

- Muchos emojis en comentarios de sección (// 📷 Foto de tu mascota) son redundantes porque el `_sectionTitle("Foto de tu mascota")` ya lo dice.

```

142     children: [
143         // 📷 Foto de tu mascota
144         _sectionTitle("Foto de tu mascota"),
145         _card(
146             child: Column(
147                 crossAxisAlignment: CrossAxisAlignment.center,
148                 children: [
149                     // 🖼️ Previsualización de imágenes adicionales (Miniaturas con .skip(1))
150                     // Esto solo se ejecuta si hay más de una foto.
151                     if (vm.fotos.length > 1)
152                     Padding(

```

- Fail Fast.

Se aplica bien en los botones de Cámara/Galería:

if (picked == null) return;

```

392     if (picked == null) return;
393
394     try {
395         ScaffoldMessenger.of(context).showSnackBar(
396             const SnackBar(
397                 content: Text("Analizando imagen... 🌀"),
398                 backgroundColor: Colors.blueAccent,
399                 duration: Duration(seconds: 2),
400             ), // SnackBar
401         );
402
403         dynamic imagenParaVM;
404
405         if (kIsWeb) {
406             // En Web -> usamos bytes del archivo
407             imagenParaVM = await picked.readAsBytes();
408         } else {
409             // En Android/iOS -> usamos File normal
410             imagenParaVM = File(picked.path);
411         }
412
413         final url = await vm.subirFoto(imagenParaVM);
414
415         vm.agregarFoto(url);
416
417         ScaffoldMessenger.of(context).showSnackBar(
418             const SnackBar(
419                 content: Text(
420                     "✅ Imagen válida detectada y subida correctamente.",

```

Esto es fail fast correcto.

Mal aplicado, anidado en vez de retorno temprano: línea 229 (if (picked != null) { ... } envuelve todo el bloque en lugar de hacer if (picked == null) return;)

```
430     },
431     },
432     },
433     },
434     icon: const Icon(Icons.camera_alt_outlined, size: 18),
435     label: const Text("Cámara"),
436     style: ElevatedButton.styleFrom(
437       foregroundColor: const Color(0xFF2563EB),
438       backgroundColor: const Color(0xFFEFF6FF),
439       elevation: 0,
440       shape: RoundedRectangleBorder(
441         borderRadius: BorderRadius.circular(12),
442       ), // RoundedRectangleBorder
443       padding: const EdgeInsets.symmetric(
444         horizontal: 16,
445         vertical: 10,
446       ), // EdgeInsets.symmetric
447     ),
448   ), // ElevatedButton.icon
449   const SizedBox(width: 12),
450   // Galería (foto)
451   ElevatedButton.icon(
452     // ... (onPressed y estilo)
453     onPressed: () async {
454       final picker = ImagePicker();
455       final picked = await picker.pickImage(
456         source: ImageSource.gallery,
457       );
458       if (picked == null) return;
```

Buen fail fast de validación de negocio: la validación if (vm.fotos.isEmpty) { ...; return; } antes de

intentar `vm.formKeyPaso1.currentState!.validate()` en el botón "Continuar" del Paso 1.

```
600      onTap: () {
601        if (vm.fotos.isEmpty) {
602          ScaffoldMessenger.of(context).showSnackBar(
603            const SnackBar(
604              content: Text(
605                "Debes agregar al menos una foto de la mascota",
606              ), // Text
607            ), // SnackBar
608          );
609          return;
610        }
611        if (vm.formKeyPaso1.currentState!.validate()) {
612          vm.siguientePaso();
613        }
614      },
615    ),
616  ],
```

- Evita los números mágicos.

El total de pasos del wizard (3) está hardcodeado dos veces: "Paso $\{vm.paso + 1\}$ de 3" y $(vm.paso + 1) / 3$. Si agregas un paso, hay que acordarse de cambiarlo en dos sitios.

```
46      Text(
47        "Paso  $\{vm.paso + 1\}$  de 3",
48        style: const TextStyle(color: Color(0xFF6366F1), fontSize: 14),
49      ), // Text
50      const SizedBox(height: 8),
51      // Barra de progreso con degradado
52      Container(
53        height: 6,
54        width: MediaQuery.of(context).size.width * 0.6,
55        decoration: BoxDecoration(
56          borderRadius: BorderRadius.circular(10),
57          color: Colors.grey[200],
58        ), // BoxDecoration
59      child: LayoutBuilder(
60        builder: (context, constraints) {
61          final width = (vm.paso + 1) / 3 * constraints.maxWidth;
62          return AnimatedContainer(
```

Colores repetidos como literales: `Color(0xFF6366F1)`, `Color(0xFF2563EB)`,

Color(0xFF4D9EF6), Color(0xFFFFB300), etc., aparecen sueltos en varios widgets en lugar de definirse una sola vez (ej. en un AppColors o Theme).

```
47         "Paso ${vm.paso + 1} de 3",
48         style: const TextStyle(color: Color(0xFF6366F1), fontSize: 14),
49     ), // Text
50
51     gradient: const LinearGradient(
52       colors: [Color(0xFF6366F1), Color(0xFF2563EB)],
53     ), // LinearGradient
54
55     fontweight: FontWeight.bold,
56     color: Color(0xFF4D9EF6),
57   ), // TextStyle
58
59   style: ElevatedButtonStyle.from(
60     backgroundColor: const Color(0xFF4D9EF6),
61     foregroundColor: Colors.white,
62     shape: RoundedRectangleBorder(
63       borderRadius:
64         Icons.map_outlined,
65       color: Color(0xFF4D9EF6),
66     ), // Icon
```

Duraciones mágicas: (Duration(milliseconds: 400)), (Duration(seconds: 2), repetida 3 veces)

```
62   return AnimatedContainer(
63     duration: const Duration(milliseconds: 400),
64     width: width,
65   ), // SnackBar
66
67   backgroundColor: Colors.blueAccent,
68   duration: Duration(seconds: 2),
69   ), // SnackBar
70
71   backgroundColor: Colors.blueAccent,
72   duration: Duration(seconds: 2),
73   ), // SnackBar
74
75   backgroundColor: Colors.blueAccent,
76   duration: Duration(seconds: 2),
77   ), // SnackBar
```

Tamaños de imagen/ícono sueltos: (height: 80, width: 80), (height: 200), (size: 28)

```
170     height: 80,
171     width: 80,
172
173   _card(
174     child: SizedBox(
175       height: 200,
176
177     Icons.camera_alt_rounded,
178     size: 28,
179     color: Colors.grey,
```

- Utiliza buenos nombres.

Mismo valor, nombres distintos: (imagenParaEnviar)

```
238
239 dynamic imagenParaEnviar;
240
```

vs (imagenParaVM), es el mismo concepto con dos nombres diferentes.

```
403 dynamic imagenParaVM;
404
```

```
473
474 dynamic imagenParaVM;
475
```

Nombre poco descriptivo: ((f) => ..., podría ser urlFoto)

```
161
162 (f) => ClipRRect(
163   borderRadius: BorderRadius.circular(12),
164   child: Stack(
```

Nombre poco descriptivo: (String m = ..., podría ser montoFormateado)

```
943 if (r.montoRecompensa.isNotEmpty) ...[
944   (() {
945     String m = r.montoRecompensa.trim();
946     if (!m.startsWith(RegExp(r'^S/\.\s*')) {
```

- No uses variables globales.

No hay estado mutable global. El estado vive en ReporteMascotaVM vía Provider , context.watch<ReporteMascotaVM>()).

```
25 Widget build(BuildContext context) {
26   final vm = context.watch<ReporteMascotaVM>();
27
```

- Devuelve valores, no los imprimas.

Bien aplicado: no hay ningún print() ni debugPrint(). Los errores se capturan con try/catch y se muestran al usuario en un SnackBar, no se imprimen en consola.

El manejo de errores correcto, (catch (e) { ScaffoldMessenger... }) como ejemplo de que no hay print().

```
132 Widget build(BuildContext context) {  
256     ), // Text  
257     backgroundColor: Colors.green,  
258     ), // SnackBar  
259     );  
260     } catch (e) {  
261         ScaffoldMessenger.of(context).showSnackBar(  
262             SnackBar(  
263                 content: Text(  
264                     e.toString().replaceAll("Exception: ", ""),  
265                     style: const TextStyle(  
266                         fontWeight: FontWeight.bold,  
267                     ), // TextStyle  
268                 ), // Text  
269                 backgroundColor: Colors.redAccent,  
270             ), // SnackBar  
271         );  
272     }  
273 }  
274 }
```

- Utiliza los espacios en blanco para mejor lectura.

Buen uso de SizedBox para espaciar secciones. El problema real aquí no es de espacios en blanco sino de tamaño del método: el build() de Paso1Mascota mezcla construcción de UI, lógica de selección de imagen y manejo de errores en un solo método gigante. Separar la lógica de "subir foto" en un método aparte (aparte de resolver la repetición del punto 1) también haría el build() mucho más corto y fácil de leer de un vistazo.

```
376     ), // GestureDetector  
377  
378     const SizedBox(height: 16),  
379  
380     // 📷 Botones cámara y galería (Botones secundarios)  
381     Row(  
382         children: [  
383             ElevatedButton(  
384                 onPressed: () {  
385                     // ...  
386                 },  
387                 child: Text("Cámara"),  
388             ),  
389             ElevatedButton(  
390                 onPressed: () {  
391                     // ...  
392                 },  
393                 child: Text("Galería"),  
394             ),  
395         ],  
396     ),  
397     ),  
398     ),  
399     ),  
400     ),  
401     ),  
402     ),  
403     ),  
404     ),  
405     ),  
406     ),  
407     ),  
408     ),  
409     ),  
410     ),  
411     ),  
412     ),  
413     ),  
414     ),  
415     ),  
416     ),  
417     ),  
418     ),  
419     ),  
420     ),  
421     ),  
422     ),  
423     ),  
424     ),  
425     ),  
426     ),  
427     ),  
428     ),  
429     ),  
430     ),  
431     ),  
432     ),  
433     ),  
434     ),  
435     ),  
436     ),  
437     ),  
438     ),  
439     ),  
440     ),  
441     ),  
442     ),  
443     ),  
444     ),  
445     ),  
446     ),  
447     ),  
448     ),  
449     ),  
450     ),  
451     ),  
452     ),  
453     ),  
454     ),  
455     ),  
456     ),  
457     ),  
458     ),  
459     ),  
460     ),  
461     ),  
462     ),  
463     ),  
464     ),  
465     ),  
466     ),  
467     ),  
468     ),  
469     ),  
470     ),  
471     ),  
472     ),  
473     ),  
474     ),  
475     ),  
476     ),  
477     ),  
478     ),  
479     ),  
480     ),  
481     ),  
482     ),  
483     ),  
484     ),  
485     ),  
486     ),  
487     ),  
488     ),  
489     ),  
490     ),  
491     ),  
492     ),  
493     ),  
494     ),  
495     ),  
496     ),  
497     ),  
498     ),  
499     ),  
500     ),  
501     ),  
502     ),  
503     ),  
504     ),  
505     ),  
506     ),  
507     ),  
508     ),  
509     ),  
510     ),  
511     ),  
512     ),  
513     ),  
514     ),  
515     ),  
516     ),  
517     ),  
518     ),  
519     ),  
520     ),  
521     ),  
522     ),  
523     ),  
524     ),  
525     ),  
526     ),  
527     ),  
528     ),  
529     ),  
530     ),  
531     ),  
532     ),  
533     ),  
534     ),  
535     ),  
536     ),  
537     ),  
538     ),  
539     ),  
540     ),  
541     ),  
542     ),  
543     ),  
544     ),  
545     ),  
546     ),  
547     ),  
548     ),  
549     ),  
550     ),  
551     ),  
552     ),  
553     ),  
554     ),  
555     ),  
556     ),  
557     ),  
558     ),  
559     ),  
560     ),  
561     ),  
562     ),  
563     ),  
564     ),  
565     ),  
566     ),  
567     ),  
568     ),  
569     ),  
570     ),  
571     ),  
572     ),  
573     ),  
574     ),  
575     ),  
576     ),  
577     ),  
578     ),  
579     ),  
580     ),  
581     ),  
582     ),  
583     ),  
584     ),  
585     ),  
586     ),  
587     ),  
588     ),  
589     ),  
590     ),  
591     ),  
592     ),  
593     ),  
594     ),  
595     ),  
596     ),  
597     ),  
598     ),  
599     ),  
600     ),  
601     ),  
602     ),  
603     ),  
604     ),  
605     ),  
606     ),  
607     ),  
608     ),  
609     ),  
610     ),  
611     ),  
612     ),  
613     ),  
614     ),  
615     ),  
616     ),  
617     ),  
618     ),  
619     ),  
620     ),  
621     ),  
622     ),  
623     ),  
624     ),  
625     ),  
626     ),  
627     ),  
628     ),  
629     ),  
630     ),  
631     ),  
632     ),  
633     ),  
634     ),  
635     ),  
636     ),  
637     ),  
638     ),  
639     ),  
640     ),  
641     ),  
642     ),  
643     ),  
644     ),  
645     ),  
646     ),  
647     ),  
648     ),  
649     ),  
650     ),  
651     ),  
652     ),  
653     ),  
654     ),  
655     ),  
656     ),  
657     ),  
658     ),  
659     ),  
660     ),  
661     ),  
662     ),  
663     ),  
664     ),  
665     ),  
666     ),  
667     ),  
668     ),  
669     ),  
670     ),  
671     ),  
672     ),  
673     ),  
674     ),  
675     ),  
676     ),  
677     ),  
678     ),  
679     ),  
680     ),  
681     ),  
682     ),  
683     ),  
684     ),  
685     ),  
686     ),  
687     ),  
688     ),  
689     ),  
690     ),  
691     ),  
692     ),  
693     ),  
694     ),  
695     ),  
696     ),  
697     ),  
698     ),  
699     ),  
700     ),  
701     ),  
702     ),  
703     ),  
704     ),  
705     ),  
706     ),  
707     ),  
708     ),  
709     ),  
710     ),  
711     ),  
712     ),  
713     ),  
714     ),  
715     ),  
716     ),  
717     ),  
718     ),  
719     ),  
720     ),  
721     ),  
722     ),  
723     ),  
724     ),  
725     ),  
726     ),  
727     ),  
728     ),  
729     ),  
730     ),  
731     ),  
732     ),  
733     ),  
734     ),  
735     ),  
736     ),  
737     ),  
738     ),  
739     ),  
740     ),  
741     ),  
742     ),  
743     ),  
744     ),  
745     ),  
746     ),  
747     ),  
748     ),  
749     ),  
750     ),  
751     ),  
752     ),  
753     ),  
754     ),  
755     ),  
756     ),  
757     ),  
758     ),  
759     ),  
760     ),  
761     ),  
762     ),  
763     ),  
764     ),  
765     ),  
766     ),  
767     ),  
768     ),  
769     ),  
770     ),  
771     ),  
772     ),  
773     ),  
774     ),  
775     ),  
776     ),  
777     ),  
778     ),  
779     ),  
780     ),  
781     ),  
782     ),  
783     ),  
784     ),  
785     ),  
786     ),  
787     ),  
788     ),  
789     ),  
790     ),  
791     ),  
792     ),  
793     ),  
794     ),  
795     ),  
796     ),  
797     ),  
798     ),  
799     ),  
800     ),  
801     ),  
802     ),  
803     ),  
804     ),  
805     ),  
806     ),  
807     ),  
808     ),  
809     ),  
810     ),  
811     ),  
812     ),  
813     ),  
814     ),  
815     ),  
816     ),  
817     ),  
818     ),  
819     ),  
820     ),  
821     ),  
822     ),  
823     ),  
824     ),  
825     ),  
826     ),  
827     ),  
828     ),  
829     ),  
830     ),  
831     ),  
832     ),  
833     ),  
834     ),  
835     ),  
836     ),  
837     ),  
838     ),  
839     ),  
840     ),  
841     ),  
842     ),  
843     ),  
844     ),  
845     ),  
846     ),  
847     ),  
848     ),  
849     ),  
850     ),  
851     ),  
852     ),  
853     ),  
854     ),  
855     ),  
856     ),  
857     ),  
858     ),  
859     ),  
860     ),  
861     ),  
862     ),  
863     ),  
864     ),  
865     ),  
866     ),  
867     ),  
868     ),  
869     ),  
870     ),  
871     ),  
872     ),  
873     ),  
874     ),  
875     ),  
876     ),  
877     ),  
878     ),  
879     ),  
880     ),  
881     ),  
882     ),  
883     ),  
884     ),  
885     ),  
886     ),  
887     ),  
888     ),  
889     ),  
890     ),  
891     ),  
892     ),  
893     ),  
894     ),  
895     ),  
896     ),  
897     ),  
898     ),  
899     ),  
900     ),  
901     ),  
902     ),  
903     ),  
904     ),  
905     ),  
906     ),  
907     ),  
908     ),  
909     ),  
910     ),  
911     ),  
912     ),  
913     ),  
914     ),  
915     ),  
916     ),  
917     ),  
918     ),  
919     ),  
920     ),  
921     ),  
922     ),  
923     ),  
924     ),  
925     ),  
926     ),  
927     ),  
928     ),  
929     ),  
930     ),  
931     ),  
932     ),  
933     ),  
934     ),  
935     ),  
936     ),  
937     ),  
938     ),  
939     ),  
940     ),  
941     ),  
942     ),  
943     ),  
944     ),  
945     ),  
946     ),  
947     ),  
948     ),  
949     ),  
950     ),  
951     ),  
952     ),  
953     ),  
954     ),  
955     ),  
956     ),  
957     ),  
958     ),  
959     ),  
960     ),  
961     ),  
962     ),  
963     ),  
964     ),  
965     ),  
966     ),  
967     ),  
968     ),  
969     ),  
970     ),  
971     ),  
972     ),  
973     ),  
974     ),  
975     ),  
976     ),  
977     ),  
978     ),  
979     ),  
980     ),  
981     ),  
982     ),  
983     ),  
984     ),  
985     ),  
986     ),  
987     ),  
988     ),  
989     ),  
990     ),  
991     ),  
992     ),  
993     ),  
994     ),  
995     ),  
996     ),  
997     ),  
998     ),  
999     ),  
1000     ),  
1001     ),  
1002     ),  
1003     ),  
1004     ),  
1005     ),  
1006     ),  
1007     ),  
1008     ),  
1009     ),  
1010     ),  
1011     ),  
1012     ),  
1013     ),  
1014     ),  
1015     ),  
1016     ),  
1017     ),  
1018     ),  
1019     ),  
1020     ),  
1021     ),  
1022     ),  
1023     ),  
1024     ),  
1025     ),  
1026     ),  
1027     ),  
1028     ),  
1029     ),  
1030     ),  
1031     ),  
1032     ),  
1033     ),  
1034     ),  
1035     ),  
1036     ),  
1037     ),  
1038     ),  
1039     ),  
1040     ),  
1041     ),  
1042     ),  
1043     ),  
1044     ),  
1045     ),  
1046     ),  
1047     ),  
1048     ),  
1049     ),  
1050     ),  
1051     ),  
1052     ),  
1053     ),  
1054     ),  
1055     ),  
1056     ),  
1057     ),  
1058     ),  
1059     ),  
1060     ),  
1061     ),  
1062     ),  
1063     ),  
1064     ),  
1065     ),  
1066     ),  
1067     ),  
1068     ),  
1069     ),  
1070     ),  
1071     ),  
1072     ),  
1073     ),  
1074     ),  
1075     ),  
1076     ),  
1077     ),  
1078     ),  
1079     ),  
1080     ),  
1081     ),  
1082     ),  
1083     ),  
1084     ),  
1085     ),  
1086     ),  
1087     ),  
1088     ),  
1089     ),  
1090     ),  
1091     ),  
1092     ),  
1093     ),  
1094     ),  
1095     ),  
1096     ),  
1097     ),  
1098     ),  
1099     ),  
1100     ),  
1101     ),  
1102     ),  
1103     ),  
1104     ),  
1105     ),  
1106     ),  
1107     ),  
1108     ),  
1109     ),  
1110     ),  
1111     ),  
1112     ),  
1113     ),  
1114     ),  
1115     ),  
1116     ),  
1117     ),  
1118     ),  
1119     ),  
1120     ),  
1121     ),  
1122     ),  
1123     ),  
1124     ),  
1125     ),  
1126     ),  
1127     ),  
1128     ),  
1129     ),  
1130     ),  
1131     ),  
1132     ),  
1133     ),  
1134     ),  
1135     ),  
1136     ),  
1137     ),  
1138     ),  
1139     ),  
1140     ),  
1141     ),  
1142     ),  
1143     ),  
1144     ),  
1145     ),  
1146     ),  
1147     ),  
1148     ),  
1149     ),  
1150     ),  
1151     ),  
1152     ),  
1153     ),  
1154     ),  
1155     ),  
1156     ),  
1157     ),  
1158     ),  
1159     ),  
1160     ),  
1161     ),  
1162     ),  
1163     ),  
1164     ),  
1165     ),  
1166     ),  
1167     ),  
1168     ),  
1169     ),  
1170     ),  
1171     ),  
1172     ),  
1173     ),  
1174     ),  
1175     ),  
1176     ),  
1177     ),  
1178     ),  
1179     ),  
1180     ),  
1181     ),  
1182     ),  
1183     ),  
1184     ),  
1185     ),  
1186     ),  
1187     ),  
1188     ),  
1189     ),  
1190     ),  
1191     ),  
1192     ),  
1193     ),  
1194     ),  
1195     ),  
1196     ),  
1197     ),  
1198     ),  
1199     ),  
1200     ),  
1201     ),  
1202     ),  
1203     ),  
1204     ),  
1205     ),  
1206     ),  
1207     ),  
1208     ),  
1209     ),  
1210     ),  
1211     ),  
1212     ),  
1213     ),  
1214     ),  
1215     ),  
1216     ),  
1217     ),  
1218     ),  
1219     ),  
1220     ),  
1221     ),  
1222     ),  
1223     ),  
1224     ),  
1225     ),  
1226     ),  
1227     ),  
1228     ),  
1229     ),  
1230     ),  
1231     ),  
1232     ),  
1233     ),  
1234     ),  
1235     ),  
1236     ),  
1237     ),  
1238     ),  
1239     ),  
1240     ),  
1241     ),  
1242     ),  
1243     ),  
1244     ),  
1245     ),  
1246     ),  
1247     ),  
1248     ),  
1249     ),  
1250     ),  
1251     ),  
1252     ),  
1253     ),  
1254     ),  
1255     ),  
1256     ),  
1257     ),  
1258     ),  
1259     ),  
1260     ),  
1261     ),  
1262     ),  
1263     ),  
1264     ),  
1265     ),  
1266     ),  
1267     ),  
1268     ),  
1269     ),  
1270     ),  
1271     ),  
1272     ),  
1273     ),  
1274     ),  
1275     ),  
1276     ),  
1277     ),  
1278     ),  
1279     ),  
1280     ),  
1281     ),  
1282     ),  
1283     ),  
1284     ),  
1285     ),  
1286     ),  
1287     ),  
1288     ),  
1289     ),  
1290     ),  
1291     ),  
1292     ),  
1293     ),  
1294     ),  
1295     ),  
1296     ),  
1297     ),  
1298     ),  
1299     ),  
1300     ),  
1301     ),  
1302     ),  
1303     ),  
1304     ),  
1305     ),  
1306     ),  
1307     ),  
1308     ),  
1309     ),  
1310     ),  
1311     ),  
1312     ),  
1313     ),  
1314     ),  
1315     ),  
1316     ),  
1317     ),  
1318     ),  
1319     ),  
1320     ),  
1321     ),  
1322     ),  
1323     ),  
1324     ),  
1325     ),  
1326     ),  
1327     ),  
1328     ),  
1329     ),  
1330     ),  
1331     ),  
1332     ),  
1333     ),  
1334     ),  
1335     ),  
1336     ),  
1337     ),  
1338     ),  
1339     ),  
1340     ),  
1341     ),  
1342     ),  
1343     ),  
1344     ),  
1345     ),  
1346     ),  
1347     ),  
1348     ),  
1349     ),  
1350     ),  
1351     ),  
1352     ),  
1353     ),  
1354     ),  
1355     ),  
1356     ),  
1357     ),  
1358     ),  
1359     ),  
1360     ),  
1361     ),  
1362     ),  
1363     ),  
1364     ),  
1365     ),  
1366     ),  
1367     ),  
1368     ),  
1369     ),  
1370     ),  
1371     ),  
1372     ),  
1373     ),  
1374     ),  
1375     ),  
1376     ),  
1377     ),  
1378     ),  
1379     ),  
1380     ),  
1381     ),  
1382     ),  
1383     ),  
1384     ),  
1385     ),  
1386     ),  
1387     ),  
1388     ),  
1389     ),  
1390     ),  
1391     ),  
1392     ),  
1393     ),  
1394     ),  
1395     ),  
1396     ),  
1397     ),  
1398     ),  
1399     ),  
1400     ),  
1401     ),  
1402     ),  
1403     ),  
1404     ),  
1405     ),  
1406     ),  
1407     ),  
1408     ),  
1409     ),  
1410     ),  
1411     ),  
1412     ),  
1413     ),  
1414     ),  
1415     ),  
1416     ),  
1417     ),  
1418     ),  
1419     ),  
1420     ),  
1421     ),  
1422     ),  
1423     ),  
1424     ),  
1425     ),  
1426     ),  
1427     ),  
1428     ),  
1429     ),  
1430     ),  
1431     ),  
1432     ),  
1433     ),  
1434     ),  
1435     ),  
1436     ),  
1437     ),  
1438     ),  
1439     ),  
1440     ),  
1441     ),  
1442     ),  
1443     ),  
1444     ),  
1445     ),  
1446     ),  
1447     ),  
1448     ),  
1449     ),  
1450     ),  
1451     ),  
1452     ),  
1453     ),  
1454     ),  
1455     ),  
1456     ),  
1457     ),  
1458     ),  
1459     ),  
1460     ),  
1461     ),  
1462     ),  
1463     ),  
1464     ),  
1465     ),  
1466     ),  
1467     ),  
1468     ),  
1469     ),  
1470     ),  
1471     ),  
1472     ),  
1473     ),  
1474     ),  
1475     ),  
1476     ),  
1477     ),  
1478     ),  
1479     ),  
1480     ),  
1481     ),  
1482     ),  
1483     ),  
1484     ),  
1485     ),  
1486     ),  
1487     ),  
1488     ),  
1489     ),  
1490     ),  
1491     ),  
1492     ),  
1493     ),  
1494     ),  
1495     ),  
1496     ),  
1497     ),  
1498     ),  
1499     ),  
1500     ),  
1501     ),  
1502     ),  
1503     ),  
1504     ),  
1505     ),  
1506     ),  
1507     ),  
1508     ),  
1509     ),  
1510     ),  
1511     ),  
1512     ),  
1513     ),  
1514     ),  
1515     ),  
1516     ),  
1517     ),  
1518     ),  
1519     ),  
1520     ),  
1521     ),  
1522     ),  
1523     ),  
1524     ),  
1525     ),  
1526     ),  
1527     ),  
1528     ),  
1529     ),  
1530     ),  
1531     ),  
1532     ),  
1533     ),  
1534     ),  
1535     ),  
1536     ),  
1537     ),  
1538     ),  
1539     ),  
1540     ),  
1541     ),  
1542     ),  
1543     ),  
1544     ),  
1545     ),  
1546     ),  
1547     ),  
1548     ),  
1549     ),  
1550     ),  
1551     ),  
1552     ),  
1553     ),  
1554     ),  
1555     ),  
1556     ),  
1557     ),  
1558     ),  
1559     ),  
1560     ),  
1561     ),  
1562     ),  
1563     ),  
1564     ),  
1565     ),  
1566     ),  
1567     ),  
1568     ),  
1569     ),  
1570     ),  
1571     ),  
1572     ),  
1573     ),  
1574     ),  
1575     ),  
1576     ),  
1577     ),  
1578     ),  
1579     ),  
1580     ),  
1581     ),  
1582     ),  
1583     ),  
1584     ),  
1585     ),  
1586     ),  
1587     ),  
1588     ),  
1589     ),  
1590     ),  
1591     ),  
1592     ),  
1593     ),  
1594     ),  
1595     ),  
1596     ),  
1597     ),  
1598     ),  
1599     ),  
1600     ),  
1601     ),  
1602     ),  
1603     ),  
1604     ),  
1605     ),  
1606     ),  
1607     ),  
1608     ),  
1609     ),  
1610     ),  
1611     ),  
1612     ),  
1613     ),  
1614     ),  
1615     ),  
1616     ),  
1617     ),  
1618     ),  
1619     ),  
1620     ),  
1621     ),  
1622     ),  
1623     ),  
1624     ),  
1625     ),  
1626     ),  
1627     ),  
1628     ),  
1629     ),  
1630     ),  
1631     ),  
1632     ),  
1633     ),  
1634     ),  
1635     ),  
1636     ),  
1637     ),  
1638     ),  
1639     ),  
1640     ),  
1641     ),  
1642     ),  
1643     ),  
1644     ),  
1645     ),  
1646     ),  
1647     ),  
1648     ),  
1649     ),  
1650     ),  
1651     ),  
1652     ),  
1653     ),  
1654     ),  
1655     ),  
1656     ),  
1657     ),  
1658     ),  
1659     ),  
1660     ),  
1661     ),  
1662     ),  
1663     ),  
1664     ),  
1665     ),  
1666     ),  
1667     ),  
1668     ),  
1669     ),  
1670     ),  
1671     ),  
1672     ),  
1673     ),  
1674     ),  
1675     ),  
1676     ),  
1677     ),  
1678     ),  
1679     ),  
1680     ),  
1681     ),  
1682     ),  
1683     ),  
1684     ),  
1685     ),  
1686     ),  
1687     ),  
1688     ),  
1689     ),  
1690     ),  
1691     ),  
1692     ),  
1693     ),  
1694     ),  
1695     ),  
1696     ),  
1697     ),  
1698     ),  
1699     ),  
1700     ),  
1701     ),  
1702     ),  
1703     ),  
1704     ),  
1705     ),  
1706     ),  
1707     ),  
1708     ),  
1709     ),  
1710     ),  
1711     ),  
1712     ),  
1713     ),  
1714     ),  
1715     ),  
1716     ),  
1717     ),  
1718     ),  
1719     ),  
1720     ),  
1721     ),  
1722     ),  
1723     ),  
1724     ),  
1725     ),  
1726     ),  
1727     ),  
1728     ),  
1729     ),  
1730     ),  
1731     ),  
1732     ),  
1733     ),  
1734     ),  
1735     ),  
1736     ),  
1737     ),  
1738     ),  
1739     ),  
1740     ),  
1741     ),  
1742     ),  
1743     ),  
1744     ),  
1745     ),  
174
```

```
541     ),  
542     const SizedBox(height: 12),  
543     DropdownButtonFormField<String>(  
544         decoration: _inputDecoration("Tipo de mascota"),
```

```
567     ),  
568     const SizedBox(height: 24),  
569
```

```
131 @override  
132 Widget build(BuildContext context) {  
133     final vm = context.watch<ReporteMascotaVM>();  
134     final bool fotoPrincipalDisponible = vm.fotos.isNotEmpty;  
135  
136     return Form(  
137         key: vm.formKeyPaso1,  
138         child: SingleChildScrollView(  
139             padding: const EdgeInsets.all(20),  
140             child: Column(  
141                 crossAxisAlignment: CrossAxisAlignment.start,  
142                 children: [  
143                     // 📷 Foto de tu mascota  
144                     _sectionTitle("Foto de tu mascota"),  
145                     _card(  
146                         child: Column(  
147                             crossAxisAlignment: CrossAxisAlignment.center,  
148                             children: [  
149                                 // 🖼️ Previsualización de imágenes adicionales (Miniaturas con .skip(
/>150                                 // Esto solo se ejecuta si hay más de una foto.  
151                                 if (vm.fotos.length > 1)
```

```
132   Widget build(BuildContext context) {  
133     // Row  
595     mainAxisAlignment: MainAxisAlignment.spaceBetween,  
596     children: [  
597       _outlinedNavButton("Atrás", onTap: vm.pasoAnterior),  
598       _gradientNavButton(  
599         "Continuar",  
600         onTap: () {  
601           if (vm.fotos.isEmpty) {  
602             ScaffoldMessenger.of(context).showSnackBar(  
603               const SnackBar(  
604                 content: Text(  
605                   "Debes agregar al menos una foto de la mascota",  
606                 ), // Text  
607               ), // SnackBar  
608             );  
609             return;  
610           }  
611           if (vm.formKeyPaso1.currentState!.validate()) {  
612             vm.siguientePaso();  
613           }  
614         },  
615       ),  
616     ],  
617   ), // Row  
618 ],  
619 ), // Column  
620 ), // SingleChildScrollView  
621 ); // Form  
622 }
```

Ahora código corregido:

No te repitas

Se unificó en una sola función (_seleccionarYSubirFoto) la lógica que antes estaba copiada 3 veces:

```
48 Future<void> _seleccionarYSubirFoto(  
49 BuildContext context,  
50 ReporteMascotaVM vm,  
51 ImageSource fuente,  
52 ) async {  
53 final picker = ImagePicker();  
54 final imagenSeleccionada = await picker.pickImage(source: fuente);  
55 if (imagenSeleccionada == null) return; // Fail fast: el usuario canceló.  
56  
57 if (!context.mounted) return;  
58 ScaffoldMessenger.of(context).showSnackBar(  
59 const SnackBar(  
60 content: Text("Analizando imagen... 🧠"),  
61 backgroundColor: Colors.blueAccent,  
62 duration: _ReporteEstilo.duracionSnackAnalizando,  
63 ), // SnackBar  
64 );  
65  
66 try {  
67 // En Web el picker no da un path de archivo válido, así que se usan bytes.  
68 final dynamic imagenParaVM = kIsWeb  
69 ? await imagenSeleccionada.readAsBytes()  
70 : File(imagenSeleccionada.path);  
71  
72 final url = await vm.subirFoto(imagenParaVM);  
73 vm.agregarFoto(url);  
74  
75 if (!context.mounted) return;  
76 ScaffoldMessenger.of(context).showSnackBar(  
77 const SnackBar(  
78 content: Text("Foto subida exitosamente"),  
79 backgroundColor: Colors.green, duration: Duration(seconds: 2),  
80 ), // SnackBar  
81 );  
82 } catch (e) {  
83 if (!context.mounted) return;  
84 ScaffoldMessenger.of(context).showSnackBar(  
85 const SnackBar(  
86 content: Text("Error al subir foto: $e"),  
87 backgroundColor: Colors.red, duration: Duration(seconds: 2),  
88 ), // SnackBar  
89 );  
90 }  
91 }
```

```

74
75     if (!context.mounted) return;
76     ScaffoldMessenger.of(context).showSnackBar(
77       const SnackBar(
78         content: Text("✅ Imagen válida detectada y subida correctamente."),
79         backgroundColor: Colors.green,
80       ), // SnackBar
81     );
82   } catch (e) {
83     if (!context.mounted) return;
84     ScaffoldMessenger.of(context).showSnackBar(
85       SnackBar(
86         content: Text(
87           e.toString().replaceAll("Exception: ", ""),
88           style: const TextStyle(fontWeight: FontWeight.bold),
89         ), // Text
90         backgroundColor: Colors.redAccent,
91       ), // SnackBar
92     );
93   }
94 }

```

Uso en imagen principal:

Uso en botón Cámara:

Uso en botón Galería:

onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery), // imagen principal

onPressed: () => _seleccionarYSubirFoto(context, vm, ImageSource.camera), // botón Cámara

onPressed: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery), // botón Galería

```

294
295     onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
296     child: Container(
297       width: double.infinity,

```

```

391     onPressed: () =>
392       _seleccionarYSubirFoto(context, vm, ImageSource.camera),
393     icon: const Icon(Icons.camera_alt_outlined, size: 18),

```

```

406     ElevatedButton.icon(
407       onPressed: () =>
408         _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
409       icon: const Icon(Icons.photo_library_outlined, size: 18),

```

Comenta lo necesario

Se quitaron los comentarios de relleno (como el que decía que faltaban funciones "por espacio") y los que solo repetían lo que ya decía el código. Se dejaron solo los que explican una decisión, como por qué en Web se usan bytes en vez de un archivo.

// ANTES:

// Definiciones auxiliares (_sectionTitle, _card, etc.) no incluidas aquí por espacio,

// pero son necesarias para que el código compile.

// AHORA: (se quitó por completo, no era necesario)

Se dejó porque explica una decisión real (comentario sobre por qué en Web se usan bytes en vez de archivo).

```
66      try {
67          // En Web el picker no da un path de archivo válido, así que se usan bytes.
68          final dynamic imagenParaVM = kIsWeb
69              ? await imagenSeleccionada.readAsBytes()
70              : File(imagenSeleccionada.path);
71      }
```

Fail Fast

- Retorno inmediato al cancelar selección de imagen:

```
54      final imagenSeleccionada = await picker.pickImage(source: fuente);
55      if (imagenSeleccionada == null) return; // Fail fast: el usuario canceló.
56  }
```

- Retorno inmediato al fallar/cancelar guardado del reporte: (if (!ok || !context.mounted) return;)

```
796      : () async {
797          final ok = await vm.guardarReporte();
798          if (!ok || !context.mounted) return; // Fail fast.
799      }
```

Evita los números mágicos

Clase de constantes completa:

```
15 class _ReporteEstilo {
16     _ReporteEstilo._();
17
18     // Colores
19     static const Color colorPrimario = Color(0xFF6366F1);
20     static const Color colorSecundario = Color(0xFF2563EB);
21     static const Color colorMapa = Color(0xFF4D9EF6);
22     static const Color colorRecompensa = Color(0xFFFFB300);
23     static const Color colorRecompensaTexto = Color(0xFFB45309);
24     static const Color colorGaleria = Color(0xFF7C3AED);
25     static const Color fondoCamara = Color(0xFFEFF6FF);
26     static const Color fondoGaleria = Color(0xFFFFF5F3FF);
27
28     // Estructura del wizard
29     static const int totalPasos = 3;
30
31     // Duraciones
32     static const Duration duracionAnimacionBarra = Duration(milliseconds: 400);
33     static const Duration duracionAnimacionBoton = Duration(milliseconds: 150);
34     static const Duration duracionSnackAnalizando = Duration(seconds: 2);
35
36     // Tamaños
37     static const double radioBordeCard = 16;
38     static const double radioBordeGrande = 20;
39     static const double tamanoMiniatura = 80;
```

TotalPasos definido:

```
28 // Estructura del wizard
29 static const int totalPasos = 3;
```

Uso de totalPasos en vez del número suelto:

// ANTES: "Paso \${vm.paso + 1} de 3" //

AHORRA: "Paso \${vm.paso + 1} de \${_ReporteEstilo.totalPasos}"

```
141 Text(
142     "Paso ${vm.paso + 1} de ${_ReporteEstilo.totalPasos}",
143     style: const TextStyle(color: _ReporteEstilo.colorPrimario, fontSize:
144
145     final anchoKelleno =
146         (vm.paso + 1) / _ReporteEstilo.totalPasos * constraints.maxWidth;
147     return AnimatedContainer(
```

Utiliza buenos nombres — mismo valor, un solo nombre consistente:

imagenParaVM unificado (antes eran dos nombres distintos):

```
67 // En web el picker no da un path de archivo valido, así que se usan bytes.
68 final dynamic imagenParaVM = kIsWeb
69   ? await imagenSeleccionada.readAsBytes()
70   : File(imagenSeleccionada.path);
71
```

urlFoto en vez de f: línea 245

```
245 (urlFoto) => ClipRect(
246   borderRadius: BorderRadius.circular(12),
247   child: Stack(
248     children: [
```

No uses variables globales

No hay línea que mostrar como "corrección" porque ya estaba bien, puedes usar como ejemplo de buena práctica donde se usa el ViewModel vía Provider (esto está en la clase Paso1Mascota, al inicio de su build()).

```
119 @override
120 Widget build(BuildContext context) {
121   final vm = context.watch<ReporteMascotaVM>();
122
123   return Scaffold(
```

Devuelve valores, no imprimas

Verificación tras await antes de actualizar UI:(mismo lugar que el fail fast del guardado)

```
797 final ok = await vm.guardarReporte();
798 if (!ok || !context.mounted) return; // Fail fast.
799
800 ScaffoldMessenger.of(context).showSnackBar(
801   const SnackBar(content: Text("✅ Reporte guardado con éxito")),
802 );
803 Navigator.pop(context);
804 }
```

Espacios en blanco / método más corto

- GestureDetector de la imagen principal, ahora limpio y corto:

```
291 GestureDetector(
292   onPressed: fotoPrincipalDisponible
293     ? () => _mostrarDialogoEliminar(context, vm, vm.fotos.first)
294     : null,
295   onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
296   child: Container(
```

Análisis por Principios de Calidad de Código

1. No te repitas (DRY)

- **Qué observé en mi código:**
Encontré varias partes donde estaba repitiendo la misma lógica. Por ejemplo, la lógica para mostrar fechas relativas estaba duplicada en `pantalla_comentarios_reporte.dart`, dentro del método `_fechaBonita`, y en `pantalla_detalle_completo.dart`, dentro del builder de preview de comentarios. En ambos casos calculaba de la misma manera el tiempo transcurrido para mostrar mensajes como “Hace X min” o “Hace X h”.
- También tenía repetida cuatro veces la navegación hacia `PantallaComentariosReporte` dentro de `pantalla_detalle_completo.dart`, además de repetir en varios métodos la expresión para determinar la colección de Firestore según el tipo de reporte.
- **Solución que apliqué:**
Creé una clase de utilidad compartida, `FechaUtils.formatearTiempoRelativo()`, para centralizar el formato de fechas. También encapsulé la navegación en el método `_navegarAComentarios()` y centralicé el acceso a la colección de Firestore mediante el getter `_coleccionPadre`.

2. Comenta donde sea necesario

- **Qué observé en mi código:**
Me di cuenta de que faltaba documentación formal en varias clases, métodos y modelos de datos. Tampoco tenía documentada la fórmula de Haversine que utilizaba para realizar el cálculo geodésico en `avistamiento_vm.dart`.
- Por otro lado, tenía algunos comentarios que no aportaban información adicional, como `// 📷 FOTO(S)` o `// Botón guardar`, ya que el código era suficientemente claro por sí mismo.

-
- **Solución que apliqué:**

Agregué documentación mediante Dartdoc (///) a las clases y métodos públicos. También documenté la fuente de la fórmula de Haversine y eliminé los comentarios redundantes que no aportaban valor.

3. Fail Fast

- **Qué observé en mi código:**
Encontré algunas operaciones que se ejecutaban sin realizar primero las validaciones necesarias. Por ejemplo, `subirFoto()` intentaba procesar y comprimir una imagen sin comprobar previamente si el archivo existía en el dispositivo.
- También `guardarAvistamiento()` podía iniciar una petición a Firestore sin ejecutar previamente `validarCampos()`. Además, en `_enviarComentario()`, si el usuario no introducía texto ni una foto, simplemente se hacía un `return` sin darle ninguna explicación.
- **Solución que apliqué:**
Agregué validaciones preventivas utilizando `archivo.existsSync()`, hice que `validarCampos()` se ejecute antes de guardar la información y añadí mensajes inmediatos en la interfaz para explicar al usuario por qué no podía continuar.

4. Evita los números mágicos

- **Qué observé en mi código:**
Tenía varios valores numéricos escritos directamente dentro de la lógica, como `0.95` para el umbral de confianza de TFLite, `9.0` para la distancia máxima de coincidencia en kilómetros, `0.5` para la similitud de imagen, `6371` para el radio terrestre, `70/80` para la calidad de compresión JPEG y `3` como límite de comentarios mostrados en la vista previa.
- **Solución que apliqué:**
Reemplacé estos valores por constantes estáticas con nombres descriptivos, como `_umbralConfianzaTflite`, `_distanciaMaxCoincidenciaKm` y `_limiteComentariosPreview`. De esta manera, es más fácil entender qué representa cada número y modificarlo posteriormente.

5. Un propósito para cada variable

- **Qué observé en mi código:**

En general, los parámetros de entrada no sufrían mutaciones indebidas, pero sí encontré algunos casos donde reutilizaba estructuras de datos de manera informal, especialmente en mapas utilizados para pasar parámetros.

- **Solución que apliqué:**

Organicé las variables para que cada una tuviera un propósito específico y único, evitando reutilizarlas para diferentes funciones o modificar su contenido innecesariamente durante su ciclo de vida.

6. Utiliza buenos nombres

- **Qué observé en mi código:**

Encontré identificadores demasiado genéricos, especialmente dentro de bucles y mapeos en `pantalla_detalle_completo.dart`. Por ejemplo, utilizaba nombres como `final a = doc.data()` para un avistamiento y `final d = doc.data()` para un comentario.

- **Solución que apliqué:**

Cambié estos nombres por otros más descriptivos y fáciles de entender, como `datosAvistamiento`, `documentoComentario`, `urlFotoPerfil` y `nombreAutor`. Así, puedo entender qué contiene cada variable sin tener que revisar primero de dónde proviene.

7. Convenciones de lenguaje

- **Qué observé en mi código:**

La mayoría de mis clases y métodos ya seguían las convenciones de Dart, utilizando PascalCase para las clases y camelCase para los métodos. Sin embargo, encontré algunas inconsistencias en el nombramiento de constantes y archivos.

- **Solución que apliqué:**

Estandaricé los nombres de las constantes y los archivos siguiendo las convenciones recomendadas por el style guide oficial de Dart y Flutter.

8. Usa espacios en blanco como apoyo de lectura

- **Qué observé en mi código:**

La organización general del código era buena, pero algunos widgets tenían estructuras demasiado extensas y no había una separación suficientemente clara entre la lógica de estado y la parte visual.

- **Solución que apliqué:**

Organicé mejor las vistas utilizando espacios, saltos de línea y comentarios de sección para separar los bloques relacionados con el estado, los helpers y los subwidgets. Esto permite identificar más fácilmente cada parte del código.

9. No uses variables globales / estáticas compartidas

- **Qué observé en mi código:**

Encontré un problema en `ComentariosViewModel`, que mantenía una referencia fija a la colección `comentarios`, sin tener en cuenta que los comentarios realmente pertenecían a un reporte específico y debían seguir una estructura anidada en Firestore.

- **Solución que apliqué:**

Refactoricé `ComentariosViewModel` para que reciba obligatoriamente `reporteId` y `tipo` mediante su constructor. De esta forma, puedo construir dinámicamente la ruta correcta de Firestore según el reporte correspondiente.

10. Los métodos deben retornar resultados, no imprimirlos

- **Qué observé en mi código:**

En `avistamiento_vm.dart` tenía varios `print()` dentro de la lógica del algoritmo de coincidencia, por ejemplo, para mostrar la distancia calculada o la similitud de las imágenes.

- **Solución que apliqué:**

Eliminé los `print()` de la lógica de negocio y los reemplacé por `debugPrint()` únicamente en los bloques `catch` destinados al manejo de excepciones. Así evito llenar la consola con mensajes innecesarios y mantengo los registros de depuración

en los lugares donde realmente son útiles.

3. Comparativa de Código: ANTES vs. DESPUÉS

Requerimiento 1: Visualización de Avistamientos

- 📄 Archivo: lib/vistamodelo/reportes/avistamiento_vm.dart
- ❌ ANTES (Código Original con números mágicos, prints y sin Fail Fast)

```
Future<String> subirFoto(File archivo) async {
  // Sin validación de existencia del archivo (Fail Fast fallido)
  final comprimido = await _comprimirImagen(archivo);
  final resultado = await ServicioTFLite.detectarAnimal(comprimido);
  final tipo = resultado["etiqueta"];

  // Número mágico 0.95 suelto
  if (resultado["confianza"] < 0.95 || (tipo != "perro" && tipo != "gato")) {
    throw Exception("❌ La imagen no parece contener un perro o gato válido.");
  }

  final uid = FirebaseAuth.instance.currentUser!.uid;
  final ref = FirebaseStorage.instance.ref().child("avistamientos").child(uid)...;
  await ref.putFile(comprimido);
  return await ref.getDownloadURL();
}

Future<void> _buscarCoincidenciaConReportes(Avistamiento av) async {
  // ...
  // Prints de consola dentro de la lógica de negocio (Violación Principio #10)
  print("📍 Distancia con ${doc.id}: ${distancia.toStringAsFixed(2)} km");
  print("🐾 Similitud con ${doc.id}: $similitud");

  // Número mágico 9.0 y 0.5 sueltos
  if (distancia > 9.0) continue;
  if (similitud >= 0.5) {
    print("✅ Avistamiento vinculado con reporte ${doc.id}");
    break;
  }
}
```

- ✅ DESPUÉS (Código Corregido y Limpio)

```
/// ViewModel para el registro y gestión de avistamientos de mascotas.
class AvistamientoVM extends ChangeNotifier {
  /// Constantes con nombres descriptivos (Principio #4)
  static const double _umbralConfianzaTflite = 0.95;
  static const int _calidadCompresionJpeg = 70;
  static const double _distanciaMaxCoincidenciaKm = 9.0;
  static const double _umbralSimilitudImagen = 0.5;
  static const double _radioTierraKm = 6371.0;

  /// Valida con TFLite que [archivo] contiene un perro o gato y lo sube a Storage.
  Future<String> subirFoto(File archivo) async {
    /// Fail Fast (Principio #3)
    if (!archivo.existsSync()) {
      throw Exception(
        'El archivo de imagen no existe. Por favor, selecciona la foto nuevamente.',
      );
    }

    final archivoComprimido = await _comprimirImagen(archivo);
    final resultado = await ServicioTFLite.detectarAnimal(archivoComprimido);

    final tipoAnimalDetectado = resultado["etiqueta"] as String;
    final confianzaDeteccion = resultado["confianza"] as double;
    final bool esAnimalValido =
      tipoAnimalDetectado == "perro" || tipoAnimalDetectado == "gato";

    if (confianzaDeteccion < _umbralConfianzaTflite || !esAnimalValido) {
      throw Exception(
        "✗ La imagen no parece contener un perro o gato válido. Intenta subir una imagen más clara.",
      );
    }

    final uid = FirebaseAuth.instance.currentUser!.uid;
    final referenciaStorage = FirebaseStorage.instance
```

```
final uid = FirebaseAuth.instance.currentUser!.uid;
final referenciaStorage = FirebaseStorage.instance
    .ref()
    .child("avistamientos")
    .child(uid)
    .child("${DateTime.now().millisecondsSinceEpoch}.jpg");

await referenciaStorage.putFile(archivoComprimido);
return await referenciaStorage.getDownloadURL();
}

/// Calculates distancia geodésica usando Haversine.
/// Fuente: https://en.wikipedia.org/wiki/Haversine\_formula (Principio #2)
double _calcularDistanciaKm(
    double latitudOrigen,
    double longitudOrigen,
    double latitudDestino,
    double longitudDestino,
) {
    final diferencialLat = _gradosARadianes(latitudDestino - latitudOrigen);
    final diferencialLon = _gradosARadianes(longitudDestino - longitudOrigen);

    final factorHaversine =
        sin(diferencialLat / 2) * sin(diferencialLat / 2) +
        cos(_gradosARadianes(latitudOrigen)) *
        cos(_gradosARadianes(latitudDestino)) *
        sin(diferencialLon / 2) *
        sin(diferencialLon / 2);

    final angulosCentrales =
        2 * atan2(sqrt(factorHaversine), sqrt(1 - factorHaversine));

    return _radioTierraKm * angulosCentrales;
}
```

```
        return _radioTierraKm * angulosCentrales;
    }

    Future<void> _buscarCoincidenciaConReportes(Avistamiento avistamientoNuevo) async {
        try {
            final reportesPerdidos = await FirebaseFirestore.instance
                .collection("reportes_mascotas")
                .where("estado", isEqualTo: "Perdido")
                .get();

            for (final documentoReporte in reportesPerdidos.docs) {
                final datosReporte = documentoReporte.data();
                final fotosReporte = List<String>.from(datosReporte["fotos"] ?? []);
                if (fotosReporte.isEmpty) continue;

                final distanciaKm = _calcularDistanciaKm(
                    avistamientoNuevo.latitud ?? 0,
                    avistamientoNuevo.longitud ?? 0,
                    (datosReporte["latitud"] ?? 0).toDouble(),
                    (datosReporte["longitud"] ?? 0).toDouble(),
                );

                // Uso de constantes nombradas sin print() (Principios #4 y #10)
                if (distanciaKm > _distanciaMaxCoincidenciaKm) continue;

                final similitudImagen = await _compararImagenes(
                    avistamientoNuevo.foto,
                    fotosReporte.first,
                );

                if (similitudImagen >= _umbralSimilitudImagen) {
                    await FirebaseFirestore.instance
                        .collection("avistamientos")
                        .doc(avistamientoNuevo.id)
                        .update({"reporteId": documentoReporte.id});

                    if (similitudImagen >= _umbralSimilitudImagen) {
                        await FirebaseFirestore.instance
                            .collection("avistamientos")
                            .doc(avistamientoNuevo.id)
                            .update({"reporteId": documentoReporte.id});
                        break;
                    }
                }
            }
        } catch (error) {
            debugPrint("▲ Error al buscar coincidencias con reportes: $error");
        }
    }
}
```

Requerimiento 2: Comentarios de Usuarios en Publicaciones

- 📄 Archivo: lib/utils/fecha_utils.dart (Nuevo módulo reutilizable)
- ✅ CÓDIGO CREADO (Elimina duplicación de lógica DRY - Principio #1)

```
/// Utilidades estáticas para el formateo de fechas en formato
/// relativo al momento actual, reutilizables en toda la aplicación.
class FechaUtils {
  FechaUtils._();

  static const int _minutosEnUnaHora = 60;
  static const int _horasEnUnDia = 24;
  static const int _diasEnUnaSemana = 7;

  /// Formatea [fecha] como texto relativo al instante actual.
  static String formatearTiempoRelativo(DateTime? fecha) {
    if (fecha == null) return '';

    final ahora = DateTime.now();
    final diferencia = ahora.difference(fecha);
    final horaFormateada =
      '${fecha.hour.toString().padLeft(2, '0')}: '
      '${fecha.minute.toString().padLeft(2, '0')}';

    if (diferencia.inMinutes < 1) return 'Ahora';
    if (diferencia.inMinutes < _minutosEnUnaHora) {
      return 'Hace ${diferencia.inMinutes} min';
    }
    if (diferencia.inHours < _horasEnUnDia) {
      return 'Hace ${diferencia.inHours} h';
    }
    if (diferencia.inDays == 1) return 'Ayer $horaFormateada';
    if (diferencia.inDays < _diasEnUnaSemana) {
      return 'Hace ${diferencia.inDays} días';
    }
    return '${fecha.day}/${fecha.month}/${fecha.year}';
  }
}
```

- 📄 Archivo: lib/vista/reportes/pantalla_detalle_completo.dart
- ❌ ANTES (Navegación duplicada 4 veces, variable a y d crípticas, formato fecha inline)

```
// Repetición inline de la lógica de fechas (DRY fallido)
String fechaTexto = '';
if (fecha != null) {
    final diff = DateTime.now().difference(fecha);
    if (diff.inMinutes < 60) {
        fechaTexto = 'Hace ${diff.inMinutes} min';
    } else if (diff.inHours < 24) {
        fechaTexto = 'Hace ${diff.inHours} h';
    } else {
        fechaTexto = '${fecha.day}/${fecha.month}/${fecha.year}';
    }
}

// Variables crípticas de una letra (Principio #6 fallido)
final a = doc.data() as Map<String, dynamic>;
final d = doc.data() as Map<String, dynamic>;

// Bloque de navegación duplicado 4 veces (Principio #1 fallido)
onPressed: () => Navigator.push(
    context,
    MaterialPageRoute(
        builder: (_) => PantallaComentariosReporte(
            reporteId: data['id'] ?? '',
            tipo: widget.tipo,
            tituloReporte: data['nombre'] ?? data['direccion'] ?? 'Reporte',
        ),
    ),
),
),
),
```

- ☒ DESPUÉS (Código Refactorizado y Estandarizado)

```
class _PantallaDetalleCompletoState extends State<PantallaDetalleCompleto> {
  // Constante nombrada (Principio #4)
  static const int _limiteComentariosPreview = 3;

  /// Navega a [PantallaComentariosReporte] centralizando el flujo (Principio #1 DRY).
  void _navegarAComentarios() {
    final data = widget.data;
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (_) => PantallaComentariosReporte(
          reporteId: data['id'] ?? '',
          tipo: widget.tipo,
          tituloReporte: data['nombre'] ?? data['direccion'] ?? 'Reporte',
        ),
      ),
    );
  }

  // ...
  // En el build del preview de avistamiento (Nombres autodescriptivos - Principio #6)
  final datosAvistamiento = doc.data() as Map<String, dynamic>;

  // En el build del preview de comentarios (Reuso de helper - Principio #1 y #6)
  final datosComentario = documentoComentario.data() as Map<String, dynamic>;
  final textoFecha = FechaUtils.formatearTiempoRelativo(fechaComentario);

  // Reutilización del método de navegación centralizado
  FloatingActionButton.extended(
    onPressed: _navegarAComentarios,
    label: const Text('Comentarios'),
  );
}
```

- 📄 Archivo: lib/vistamodelo/comentarios/comentarios_viewmodel.dart
- ❌ ANTES (Colección global hardcodeada e incorrecta)

```
class ComentariosViewModel {
  // Variable de colección fija hardcodeada (Violación Principio #9)
  final CollectionReference _ref =
    FirebaseFirestore.instance.collection('comentarios');

  Stream<List<Comentario>> obtenerComentarios() {
    return _ref.orderBy('fecha', descending: true).snapshots().map(...);
  }
}
```

-  DESPUÉS (Encapsulamiento dinámico por reporte)

```
/// ViewModel para acceder a los comentarios de un reporte o avistamiento.
class ComentariosViewModel {
    final String reporteId;
    final String tipo;

    ComentariosViewModel({
        required this.reporteId,
        required this.tipo,
    });

    String get _coleccionPadre =>
        tipo == 'reporte' ? 'reportes_mascotas' : 'avistamientos';

    CollectionReference get _comentariosRef =>
        FirebaseFirestore.instance
            .collection(_coleccionPadre)
            .doc(reporteId)
            .collection('comentarios');

    /// Retorna la lista de comentarios en tiempo real para la publicación actual.
    Stream<List<Comentario>> obtenerComentarios() {
        return _comentariosRef
            .orderBy('fecha', descending: true)
            .snapshots()
            .map((instantanea) {
                return instantanea.docs
                    .map(
                        (documento) => Comentario.fromMap(
                            documento.id,
                            documento.data() as Map<String, dynamic>,
                        ),
                    )
            })
    }
}
```

```
        .map(
            (documento) => Comentario.fromMap(
                documento.id,
                documento.data() as Map<String, dynamic>,
            ),
        )
        .toList();
    });
}
```

4. Conclusiones

- Se logró aplicar satisfactoriamente la auditoría de código basada en los 10 principios de código limpio sobre los requerimientos de Visualización de Avistamientos y Comentarios de Usuarios en Publicaciones.
- Se eliminó la duplicación de código mediante la creación de utilidades compartidas (FechaUtils) y la centralización de flujos de navegación en la interfaz de usuario.
- Se garantizó la estabilidad del sistema mediante el patrón Fail Fast, validando archivos y entradas antes de ejecutar peticiones a Firestore o Firebase Storage.
- Se erradicaron los números mágicos y los mensajes de depuración por consola (print()) en favor de constantes declarativas y manejo limpio de excepciones.

2. Conclusiones

- Se realizó una revisión del código de la aplicación móvil aplicando principios de calidad y buenas prácticas de construcción de software, identificando problemas relacionados principalmente con la duplicación de código, números mágicos, nombres poco descriptivos, variables globales y mensajes innecesarios por consola.
- Se logró reducir la duplicación de código mediante la reutilización de funcionalidades comunes. Entre las principales mejoras se encuentra la creación de FechaUtils para centralizar el formateo de fechas y la implementación del método `_navegarAComentarios()` para evitar repetir el mismo proceso de navegación en diferentes partes de la interfaz.
- La aplicación del principio Fail Fast permitió incorporar validaciones antes de ejecutar operaciones que podrían producir errores, como la comprobación de la existencia de archivos antes de procesarlos y la validación de los datos antes de realizar operaciones de almacenamiento en Firebase.
- Se mejoró la legibilidad y mantenibilidad del código mediante el reemplazo de números mágicos por constantes con nombres descriptivos. Esto permite comprender el propósito de cada valor y facilita su modificación cuando cambien los requerimientos del sistema.
- Se mejoraron los nombres de variables, parámetros y estructuras de datos para que representen claramente su propósito. Esto facilita la comprensión del código y



disminuye la posibilidad de errores durante futuras modificaciones.

- Se redujo el uso de variables globales y referencias estáticas compartidas, especialmente en la gestión de comentarios, permitiendo que el acceso a Firestore dependa del identificador y tipo de publicación correspondiente. De esta manera, se obtiene una estructura más organizada y coherente con la jerarquía de datos de la aplicación.
- Se eliminaron mensajes `print()` innecesarios dentro de la lógica de negocio y se mejoró el manejo de errores mediante excepciones y mecanismos apropiados de depuración. Esto permite separar la lógica funcional de los mensajes destinados exclusivamente al diagnóstico del sistema.